



上海交通大学学位论文



基于 Flink 的动态 Kafka-Topic 发现与订阅机制研究

姓 名：胡文杰

学 号：522031910511

导 师：任锐

学 院：计算机学院

专业名称：软件工程

申请学位层次：工学学士

2026 年 5 月

上海交通大学

学位论文原创性声明

本人郑重声明：所呈交的学位论文，是本人在导师的指导下，独立进行研究工作所取得的成果。除文中已经注明引用的内容外，本论文不包含任何其他个人或集体已经发表或撰写过的作品成果。对本文的研究做出重要贡献的个人和集体，已在文中以适当方式予以致谢。若在论文撰写过程中使用了人工智能工具，本人已遵循《上海交通大学关于在教育教学中使用 AI 的规范》，确保人工智能生成内容的应用场景、引用范围及标注方式均符合规定，并杜绝学术不端行为。本人完全知晓本声明的法律后果由本人承担。

学位论文作者签名：

日期： 年 月 日

上海交通大学

学位论文使用授权书

本人同意学校保留并向国家有关部门或机构送交论文的复印件和电子版，允许论文被查阅和借阅。

本学位论文属于：

☐ 公开论文

☐ 内部论文，保密 ☐ 1 年 / ☐ 2 年 / ☐ 3 年，过保密期后适用本授权书。

☐ 秘密论文，保密 ____ 年（不超过 10 年），过保密期后适用本授权书。

☐ 机密论文，保密 ____ 年（不超过 20 年），过保密期后适用本授权书。

（请在以上方框内选择打“√”）

学位论文作者签名：

指导教师签名：

日期： 年 月 日

日期： 年 月 日

摘 要

在云原生与事件驱动架构广泛应用的背景下，Apache Flink 与 Apache Kafka 的组合已成为实时数据流水线的常见实现方式。然而，现有 Flink Kafka Sink 多采用静态配置模式，作业启动后目标 Topic 与相关连接信息通常被固化；当下游 Kafka 拓扑因扩容、迁移、容灾切换或业务演进而发生变化时，往往需要停机修改配置并重新部署，从而增加运维成本并削弱实时链路的连续性。相比之下，社区在消费端已经探索了动态 Kafka Source，而生产端动态 Sink 的研究与实现仍相对不足。

本文围绕“基于 Flink 的动态 Kafka Topic 发现与订阅机制”展开研究。首先，从统一 Sink API、Kafka 元数据访问与静态 KafkaSink 的使用方式出发，分析其在动态写入场景下的局限；其次，结合开题报告与中期实现，提出基于正则表达式的 Topic 发现方案，以及逻辑路由与物理路由分离的设计思路；在此基础上，依托现有代码实现了 DynamicKafkaSink、DynamicKafkaSinkWriter、SingleClusterTopicMetadataService、DynamicKafkaSinkWriterState、DynamicKafkaCommittable 与 DynamicKafkaCommitter 等关键组件，并通过广播状态与 route 更新事件实现运行期动态路由。该原型能够在不重启 Flink 作业的前提下发现符合规则的 Topic、建立对应写入路径，并完成多目标写入与初步状态恢复。

目前，系统已在本地环境完成基本功能验证，验证了动态 Topic 发现、显式 route 更新与多目标写入机制的可行性。同时，本文也对当前实现中的局限进行了分析，包括元数据轮询开销、失效 route 的 writer 回收不足、Exactly-once 语义验证尚不完整等问题。后续工作将围绕元数据访问优化、资源管理、故障恢复一致性验证及与静态 Sink 的对比实验继续推进。

关键词：Flink，Kafka，动态 Topic，流处理，路由映射

ABSTRACT

With the wide adoption of cloud-native and event-driven architectures, Apache Flink and Apache Kafka have become a common combination for building real-time data pipelines. Existing Flink Kafka sinks, however, usually rely on static configurations: the target topic and connection properties are fixed when the job starts. When downstream Kafka topology changes because of expansion, migration, disaster recovery, or business evolution, the job often has to be stopped, reconfigured, and redeployed, which increases operational cost and weakens the continuity of real-time pipelines.

This thesis studies a dynamic Kafka topic discovery and subscription mechanism based on Flink. It analyzes the limitations of static `KafkaSink` usage under the unified Sink API, and designs a topic discovery approach based on regular expressions together with a separation between logical routes and physical Kafka topics. Based on the existing codebase, this work implements core components such as `DynamicKafkaSink`, `DynamicKafkaSinkWriter`, `SingleClusterTopicMetadataService`, `DynamicKafkaSinkWriterState`, `DynamicKafkaCommittable`, and `DynamicKafkaCommitter`. The prototype supports runtime route updates through broadcast state and route update events, and can discover matching topics, establish corresponding write paths, and perform multi-target writes without restarting the Flink job.

Experiments in a local environment validate the feasibility of dynamic topic discovery, explicit route updates, multi-target writes, Checkpoint coordination, and the small-scale transaction path. The thesis also discusses current limitations, including metadata polling overhead, route writer cleanup under `EXACTLY_ONCE`, and the need for more systematic fault recovery validation. Future work will focus on metadata access optimization, resource management, and more comprehensive consistency evaluation.

Key words: Flink, Kafka, dynamic topic, stream processing, route mapping

目 录

摘 要 I

ABSTRACT II

第一章 绪论 1

 1.1 研究背景 1

 1.2 研究意义 2

 1.3 国内外研究现状 3

 1.3.1 Kafka 消费模式研究现状 3

 1.3.2 Flink 流处理技术发展现状 3

 1.3.3 动态数据订阅机制相关研究 4

 1.4 本文主要研究内容 4

 1.5 论文结构安排 4

第二章 相关技术与理论基础 5

 2.1 Kafka 写入端相关机制 5

 2.1.1 Topic 元数据与管理面访问 5

 2.1.2 Topic、Partition 与写入目标绑定 5

 2.1.3 生产者事务、精准一次语义与可提交读 6

 2.2 Flink 容错与 Sink 提交机制 7

 2.2.1 Checkpoint 与动态连接器状态 7

 2.2.2 统一 Sink API 与两阶段提交 7

 2.2.3 控制流、数据流与广播状态 8

 2.3 Flink Kafka Connector 8

 2.3.1 静态 Kafka Sink 的目标绑定 8

 2.3.2 正则发现从 Source 侧到 Sink 侧的差异 9

 2.3.3 动态 Sink 需要解决连接器问题 10

 2.4 本章小结 10

第三章 系统需求分析	11
3.1 系统应用场景分析.....	11
3.2 功能需求分析.....	12
3.2.1 动态 Topic 发现需求	12
3.2.2 自动订阅需求.....	12
3.2.3 动态路由需求.....	13
3.2.4 数据处理需求.....	14
3.3 非功能需求分析.....	15
3.3.1 实时性要求	15
3.3.2 可扩展性要求.....	16
3.3.3 稳定性与容错性要求.....	16
3.4 问题建模与分析.....	17
3.5 本章小结.....	17
第四章 基于 Flink 的动态 Topic 订阅系统设计与实现.....	19
4.1 系统总体架构设计.....	19
4.2 系统模块划分.....	21
4.2.1 Topic 动态发现模块	22
4.2.2 正则匹配模块.....	22
4.2.3 自动订阅模块.....	23
4.2.4 动态路由模块.....	23
4.2.5 动态 Sink 模块.....	24
4.3 动态 Topic 发现机制设计	25
4.3.1 Kafka 元数据获取方式.....	25
4.3.2 Topic 变化检测策略	25
4.3.3 自动订阅列表更新机制	26
4.4 基于正则表达式的匹配机制.....	26
4.4.1 Pattern 设计原则.....	26
4.4.2 匹配流程与实现.....	27
4.5 自动订阅机制实现.....	27
4.5.1 动态写入目标更新策略	27
4.5.2 无重启更新机制设计.....	29
4.6 动态路由机制设计（重点）	29
4.6.1 Topic 与业务逻辑映射模型	29

4.6.2 Map 结构设计	30
4.6.3 数据分发策略.....	30
4.7 系统实现细节.....	31
4.7.1 核心类设计	31
4.7.2 数据结构设计.....	32
4.7.3 与 Flink Checkpoint 机制的集成	32
4.7.4 事务路径可运行性与 Exactly-once 语义设计	34
4.7.5 关键流程说明.....	35
4.8 本章小结.....	36
第五章 实验设计与结果分析	38
5.1 实验环境与配置.....	38
5.2 功能验证实验.....	40
5.2.1 动态 Topic 发现验证	40
5.2.2 自动订阅验证.....	41
5.2.3 动态路由验证.....	42
5.2.4 显式路由与静态基线对照验证	42
5.2.5 Checkpoint 协同验证	43
5.2.6 事务路径可运行性验证	44
5.3 非功能需求验证.....	45
5.3.1 发现实时性与 Checkpoint 协同验证.....	45
5.3.2 吞吐可接受性与小规模扩展性验证	45
5.3.3 非功能验证结果分析.....	47
5.4 对比实验与结果分析.....	47
5.4.1 静态 Kafka Sink 基线对比	47
5.4.2 Source 侧正则订阅与 Sink 侧动态写入对比	48
5.4.3 本文方案优势分析	49
5.5 本章小结.....	49
第六章 总结与展望	50
6.1 研究工作总结.....	50
6.2 存在的不足.....	50
6.3 未来工作展望.....	51

参考文献.....	52
附 录	53
1.1 实验配置样例.....	53
1.1.1 显式 route 与静态基线实验配置	53
1.1.2 发现实时性实验配置.....	53
1.2 复现实验命令.....	54
学术论文和科研成果目录	55
致 谢	56

第一章 绪论

1.1 研究背景

在云原生、微服务与事件驱动架构广泛落地的背景下，企业核心业务越来越依赖流式数据基础设施完成日志采集、行为分析、风报告警、实时推荐与在线监控等任务。**Apache Kafka** 作为分布式消息系统，凭借高吞吐、可扩展与可持久化日志模型，被广泛部署在数据接入层与消息总线层；**Apache Flink** 则通过统一流批处理引擎、有状态计算与 **Checkpoint** 容错机制，成为实时计算层的重要代表^[1-2]。二者的组合已经成为构建实时数据流水线的事实标准之一。

然而，生产环境中的实时链路并非始终保持静态。随着业务扩张与部署形态演进，**Kafka** 集群经常面临扩容、**Broker** 迁移、**Topic** 拆分合并、跨集群灰度切换与容灾演练等变化。尤其在多租户或按业务线建 **Topic** 的系统中，**Topic** 往往不是“预先固定、长期不变”的，而是会随着业务上线、租户新增、日期滚动或分域治理而动态增长。与此相对，上层 **Flink** 作业又常常承担 7×24 小时持续运行的职责，频繁停机重启不仅会增加运维负担，还可能影响数据链路连续性，带来重复写入、空窗期或恢复复杂度上升等问题。

从现有连接器能力看，**Flink** 官方 **Kafka Sink** 的主流使用方式仍以静态配置为主：在作业构建阶段指定目标 **Topic**、**bootstrap servers**、序列化器与投递语义，后续运行期间若底层目标发生变化，通常需要停止作业、更新配置、重新提交^[3-4]。这种模式在拓扑稳定场景下简单可靠，但在 **Topic** 动态变化的场景中，会明显暴露出灵活性不足的问题。特别是在业务希望“持续写入、自动扩展、无需停机”的情况下，静态 **Sink** 难以满足运维与业务上的双重要求。

如图 1-1 所示，传统方案通常要求在目标 **Sink** 变化后停止作业、修改逻辑并重新部署，其更新过程与业务写入链路强耦合；而基于模式匹配与动态路由的方案，则能够把“写到哪里”从固定配置中剥离出来，使作业在持续运行期间感知新目标并完成写入路径调整。这一对比也直观说明了本文课题的核心目标，即通过动态化机制降低停机更新对实时链路的破坏。

值得注意的是，动态能力并非完全没有先例。社区已经在 **Source** 侧推进了 **FLIP-246**^[5]，探索按模式动态发现 **Kafka Topic** 并调整订阅集合；生产端 **Sink** 方向上，**FLIP-515**^[6] 等讨论也表明，业界已经意识到“动态写入目标”是值得单独研究的问题。但与

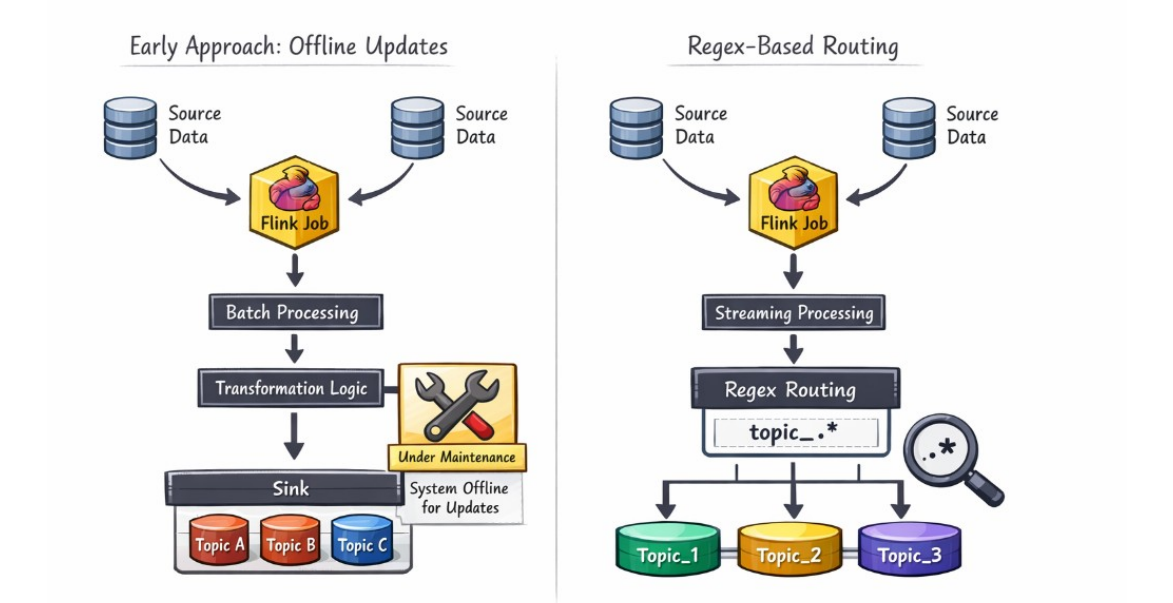


图 1-1 传统离线更新方式与基于正则的动态路由方式对比

消费端相比，Sink 侧除了要解决元数据发现，还必须处理 producer 配置、目标 Topic 绑定、writer 生命周期管理、状态快照与提交语义等一系列更复杂的问题。正是这一现实差距，构成了本文选题的直接背景。

1.2 研究意义

本课题的意义主要体现在工程实践与技术方法两个层面。

从工程实践角度看，若能够在 Flink Kafka Sink 中引入运行期动态发现与动态写入能力，则实时作业在面对 Topic 增删、规则扩展或路由调整时，就不再必须依赖人工停机修改配置。这对于持续运行的流处理业务具有直接价值：一方面可以降低人工介入频率，缩短配置变更到生效之间的等待时间；另一方面有助于减少由于作业重启而产生的链路抖动、运维窗口与恢复成本。在大规模数据平台中，这种“下游拓扑可变、上游作业不停”的能力，具有明显的工程收益。

从技术方法角度看，动态 Sink 并不是对静态 Sink 的简单封装，而是涉及元数据发现、逻辑路由与物理 Topic 映射、底层 writer 的懒创建与复用、Checkpoint 状态快照、route 级提交与恢复等多个环节的协同设计。因此，研究这一问题不仅能补足

Flink Kafka 集成中的一类能力空白，也能为“如何在统一 Sink API 约束下扩展动态行为”提供一条可复用的实现路径。特别是把动态发现与已有容错机制结合起来，对后续其它动态连接器或动态外部系统适配方案，也具有一定借鉴意义。

此外，从本科毕业设计的角度，本课题既包含对 Flink、Kafka 与连接器框架的理论学习，也包含实际代码实现、配置管理、实验验证与系统分析，具备较强的综合训练价值。

1.3 国内外研究现状

1.3.1 Kafka 消费模式研究现状

Kafka 通过 Topic、分区与消费者组等机制支撑高吞吐的消息发布与订阅^[1]。围绕 Kafka 自身的性能与运维，已有工作利用混合优化算法等对分布式消息系统配置进行自动化调优^[7]，并从分区策略、性能建模等角度改善集群负载与可观测性^[8-9]。这些研究为上层流处理连接供了可靠的存储与传输基础，但多聚焦于 Kafka 作为独立系统的优化，对“上层计算框架如何在运行期感知并跟随 Topic 变化”的耦合问题涉及相对较少。消费端语义（如组协调、分区分配）与 Sink 端生产写入在模型上并不相同，但二者共同依赖对集群元数据的理解与访问方式，相关成果对元数据获取、元数据刷新周期与客户端管理策略仍有借鉴意义。从这个角度看，Kafka 研究为本文提供的是“动态写入目标问题”的基础设施背景，而不是直接现成的解决方案。

1.3.2 Flink 流处理技术发展现状

Flink 在统一流批引擎、有状态计算与容错方面已形成较为完整的体系^[2]。学术与工业界针对流处理系统的弹性、调度与能耗等主题持续开展研究，例如面向有状态流处理系统的细粒度可扩展性^[10]、面向 Flink 流应用的能效感知调度与两层协调负载均衡^[11]等。这些工作主要改善算子并行度、资源与状态迁移，对“与外部消息系统连接器的动态适配”讨论相对有限。工业界亦有大规模流处理系统实践，如早期 Twitter 上的 Storm^[12]、LinkedIn 上有状态的 Samza^[13]等，为流式管道工程化提供参照。教程与综述性工作亦系统介绍了 Flink 的流分析能力^[14]。就工程集成而言，官方文档说明了 Flink 与 Kafka 的集成及统一 Sink API 的写入方式^[3-4]，为扩展动态写入路径提供了实现基础。可以说，Flink 本身已经具备支持动态行为所需的运行时框架，关键在于如何在连接器层面合理组织这些能力，使其既能适应变化又不破坏现有提交与恢复语义。

1.3.3 动态数据订阅机制相关研究

在连接器层面，动态数据接入已成为社区演进方向之一：消费侧 FLIP-246^[5] 等工作探索在运行期调整订阅集合；生产侧 FLIP-515^[6] 等提案讨论动态 Kafka Sink 的形态与语义。工业实践中有基于 Kafka 与 Flink 进行实时事件连接等案例^[15]。总体来看，已有工作更多从“为什么需要动态化”与“消费端如何动态订阅”角度展开，而对生产端如何把“正则或模式驱动的 Topic 发现、逻辑 route 管理、底层 writer 复用、状态快照与 route 级提交”整体组织起来，公开实现和系统验证仍相对不足。尤其在本科毕业设计这一工程实现导向的任务中，把这些思想真正落地为可运行的代码原型，仍然具有明确的研究与实践空间。本课题即面向上述缺口，侧重 Sink 侧动态 Topic 发现、逻辑路由与多目标写入的实现与说明。

1.4 本文主要研究内容

本文遵循“问题分析—方案设计—实现验证”的路径，主要工作包括：

1. 分析现有基于统一 Sink API 的 Kafka Sink 在 Topic 与生产者管理上的静态性局限^[3-4]；
2. 设计基于正则表达式的 Topic 模式、周期性元数据扫描与差异计算，形成发现—更新闭环；
3. 提出逻辑路由与物理路由分离的映射模型，结合广播状态支持运行期路由更新；
4. 基于现有代码实现 `DynamicKafkaSink`、`DynamicKafkaSinkWriter`、`SingleClusterTopicMetadataService`、`DynamicKafkaSinkWriterState`、`DynamicKafkaCommittable` 与 `DynamicKafkaCommitter` 等连接器核心组件，完成动态 route 管理、状态恢复与提交链路；
5. 以 app 模块作为实验载体，对 connector 暴露出的动态发现、显式 route 更新与静态基线能力进行验证，并据此分析当前实现的能力边界与后续优化方向。

1.5 论文结构安排

全文共分六章。第 1 章介绍背景、意义与相关工作；第 2 章概述 Kafka、Flink 及 Flink Kafka Connector 相关基础；第 3 章给出需求分析与问题建模；第 4 章阐述总体架构、模块划分及动态发现、匹配、写入与路由等关键设计；第 5 章说明实验环境、功能验证与性能对比思路；第 6 章总结全文并讨论不足与展望。

第二章 相关技术与理论基础

2.1 Kafka 写入端相关机制

2.1.1 Topic 元数据与管理面访问

Apache Kafka 以分布式、可持久化的日志抽象为核心，Broker 集群负责 Topic 数据的存储与复制，Producer 与 Consumer 通过客户端与集群交互^[1]。对于本文所研究的动态 Sink 而言，真正需要关注的不是 Kafka 的完整使用教程，而是两个与后续设计直接相关的事实：第一，Topic 是 Sink 写入目标的基本单位；第二，Topic 集合可以通过管理面接口在作业运行期间被重新查询。当前原型正是利用 `AdminClient.listTopics()` 获取集群中可见的 Topic 名称，再把这些名称封装为 `KafkaStream`，供后续正则匹配与 route 解析使用。

因此，Kafka 在本文中同时扮演两种角色：一方面，它是承接 Flink 输出结果的下游消息系统；另一方面，它又是一个需要被动态感知的外部元数据源。第 4 章中 `SingleClusterTopicMetadataService` 的职责，正是把 Kafka 管理面暴露出的 Topic 列表转换为连接器内部统一使用的 `KafkaStream/ClusterMetadata` 结构。这一点决定了本文的“动态发现”发生在 Topic 粒度，而不是 partition 粒度或消费者组订阅粒度。

图 2-1 展示了 Kafka 与 Flink 在实时数据链路中的典型位置：多类数据源先汇入 Kafka，Flink 再作为中间处理与分析引擎，将结果写入下游数据汇聚系统或业务系统。本文所研究的动态 Sink，正位于这一链路的“Flink 写出下游”位置，其目标不是替代 Kafka 或 Flink，而是在二者集成边界上增强写入端的动态适配能力。

2.1.2 Topic、Partition 与写入目标绑定

Topic 是逻辑上的消息类别，其下可划分为多个分区以实现并行与扩展。消息在分区内有序，分区之间相互独立。动态 Sink 的设计重点并不是重新实现 Kafka 的分区调度，而是决定每条记录最终交给哪个 Topic 以及哪个底层 producer/writer 实例。因此，本文把 Topic 作为动态 route 的物理落点：自动发现路径将 `stream_pattern` 命中的 Topic 展开为 `activeRouteIds`；显式路由路径则把业务 `routeId` 解析为一个或多个实际 Topic。

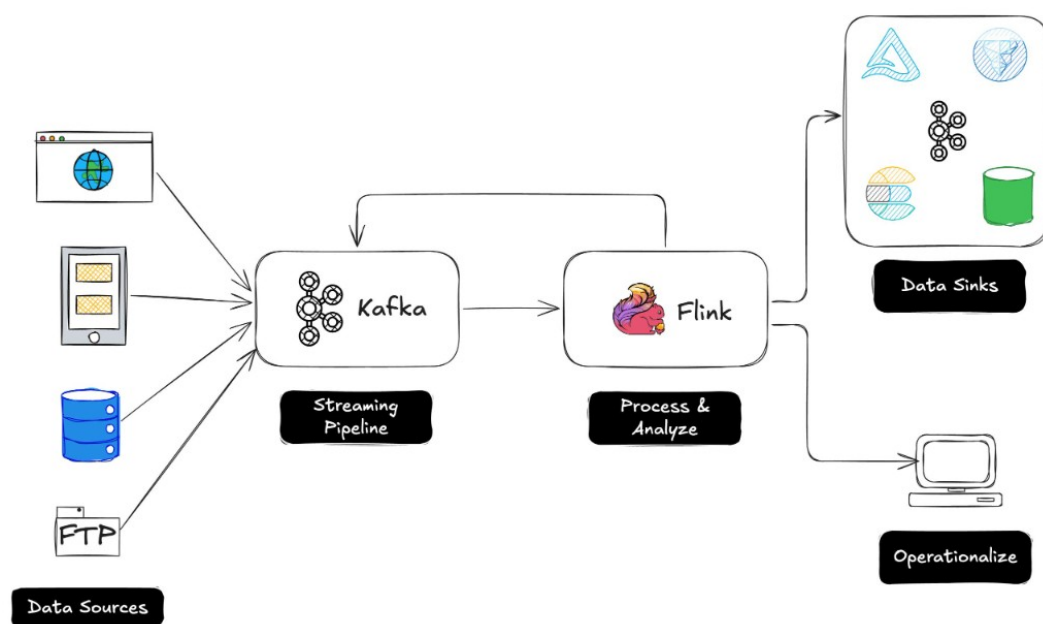


图 2-1 Kafka 与 Flink 组成的典型实时数据流水线

一旦 route 被解析为具体 Topic, 底层 `KafkaSink` 仍会按照 `Kafka producer` 的常规逻辑进行序列化、分区选择和发送确认。换言之, 本文并不改变 `Kafka` 对 `partition` 的写入语义, 而是在 `partition` 选择之前增加了一层“逻辑 route $\rightarrow$ 物理 Topic”的动态绑定。这个分层关系解释了为什么第 4 章需要在 `DynamicKafkaSinkWriter.createClusterWriter(...)` 中为每个物理 route 构造独立的底层 `writer`: 只有先把动态 route 解析成确定 Topic, 后续 `Kafka producer` 才能按原有机制完成真正发送。

2.1.3 生产者事务、精准一次语义与可提交读

本文后续涉及 `EXACTLY_ONCE` 验证, 因此需要明确 `Kafka` 写入端一致性语义的边界。`Kafka` 的普通 `producer` 只能保证记录成功发送到 `broker`, 并不能天然避免故障重试导致的重复写入。为减少重试重复, `Kafka` 引入了幂等 `producer`; 而要把一组写入作为事务提交或回滚, 则需要配置 `transactional.id`, 由 `producer` 在事务边界内写入记录, 并最终执行 `commit` 或 `abort`^[16]。

事务提交后, `Kafka` 消费端还需要通过隔离级别决定是否读取未提交数据。`isolation.level=read_committed` 表示消费者只读取已经提交的事务数据以及非事务数据; `read_uncommitted` 则可能读到尚未提交、后续也可能被回滚的记录。这一点对本文实验具有直接意义: 如果只查看 `topic offset` 或使用默认消费配置, 可能只能说明 `broker` 收到了数据, 但不足以严格证明事务数据已对下游以 `committed`

形式可见。因此，第 5 章在说明事务路径可运行性验证时，关注的是 Checkpoint 成功、事务提交未报错以及目标 Topic 出现 committed 输出，而不是把其等同于完整故障注入下的端到端精准一次证明。

对于动态 Sink 来说，Kafka 事务还带来额外复杂性：静态 Sink 通常只需要围绕固定 Topic 和固定事务前缀维护 producer；动态 Sink 则可能在运行期新增或移除 route。如果多个物理 route 共用同一事务标识，提交和恢复时容易混淆不同目标的事务边界。因此，本文实现采用 route 级事务前缀派生策略，即由全局 transactionalIdPrefix 和 routeId 生成不同物理 route 的事务前缀。第 4 章中 buildTransactionalIdPrefix(routeId)、DynamicKafkaCommittable 与 DynamicKafkaCommitter 的设计，正是建立在这一 Kafka 事务模型之上。

2.2 Flink 容错与 Sink 提交机制

2.2.1 Checkpoint 与动态连接器状态

Flink 通过 Checkpoint 与分布式快照思想实现容错^[2]。在本文场景中，Checkpoint 需要解决的不是普通算子状态如何保存这一通用问题，而是动态连接器运行期产生的 route 状态能否被恢复。动态 Sink 会在运行期间维护 activeRouteIds、cluster WritersByRouteId、显式 route 表以及底层 Kafka writer 状态。如果这些信息不进入快照，作业重启后就可能丢失已经创建的写入通道，或者无法继续提交故障前产生的事务。

因此，当前代码将动态状态组织为 route 级恢复单元：DynamicKafkaSink WriterState 记录 route 标识、目标 cluster、Topic、事务前缀、producer 配置和底层 KafkaWriterState；DynamicKafkaSink.restoreWriter(...) 再利用这些状态重建 route 对应的底层 writer。这说明后文的状态设计不是为了形式上“支持 Checkpoint”，而是为了让动态 route 在故障恢复后仍能与 Kafka 写入目标重新对齐。

2.2.2 统一 Sink API 与两阶段提交

Flink 统一 Sink API 将写入逻辑拆分为 Writer、状态快照、提交前准备和 Committer 等阶段^[3-4]。在 AT_LEAST_ONCE 语义下，Writer 通常只需保证数据被刷出到外部系统；在 EXACTLY_ONCE 语义下，Sink 还需要与 Checkpoint 配合，把某个 Checkpoint 之前的写入作为可提交单元交给 Committer。这也是本文没有简单地在用户函数里手动创建 Kafka producer 的原因：手写 producer 很难自然接入 Flink 的状态快照和提交

回调。

当前实现中, `DynamicKafkaSink` 直接实现 `TwoPhaseCommitting StatefulSink<IN, DynamicKafkaSinkWriterState, DynamicKafka Committable>`。这一选择使动态 Sink 能够在 Writer 阶段解析 route 并写入数据, 在 `snapshotState(...)` 中保存 route 级 writer 状态, 在 `prepareCommit()` 中把底层 `KafkaCommittable` 包装为带 route 信息的 `DynamicKafkaCommittable`, 最后由 `DynamicKafkaCommitter` 按 route 分组提交。第 4 章对状态和提交路径的设计, 正是对这一 API 分阶段语义的具体展开。

2.2.3 控制流、数据流与广播状态

本文的动态 route 验证依赖应用层广播状态: `DynamicJob` 读取 YAML 中的 `route_map`, 生成 `DynamicSinkEvent.update(...)` 控制事件, 再通过 `BroadcastProcessFunction` 将 route 配置写入广播状态并转发给 Sink。这一机制本身属于 Flink `DataStream` 层的能力, 但它与 connector 设计密切相关: 广播状态负责在进入 Sink 前过滤无效 `routeId`, Sink 内部则负责解释 route 更新事件并维护 `explicitRoutesById`。

因此, 本文中的控制流与数据流不是两个完全独立的系统。控制事件和普通数据事件最终都会进入 `DynamicKafkaSinkWriter.write(...)`, 再由 Writer 根据 `DynamicKafkaRouteEvent` 的语义分流。这一点为第 3 章“控制面配置必须先于数据面生效”的需求分析提供了基础, 也解释了第 4 章为什么要把 `DynamicKafkaRouteEvent` 设计为 Sink 可识别的统一事件接口。

2.3 Flink Kafka Connector

2.3.1 静态 Kafka Sink 的目标绑定

常规配置在作业构建阶段指定固定的 Topic 名称或静态列表, 运行期不随集群侧 Topic 集合变化而自动扩展。该方式实现简单、语义清晰, 适用于拓扑稳定的场景, 但在 Topic 频繁增删或按规则批量创建时, 难以避免停启作业与重复发布。本文原型中的对照组也可以理解为这种典型静态模式: 应用在作业提交前就把目标 Topic 写死到 `KafkaSink` 中, 之后任何目标变更都需要重新构建并部署作业。因此, 静态模式更适合作为本文后续实验中的基准方案, 而不是待研究问题的最终答案。

2.3.2 正则发现从 Source 侧到 Sink 侧的差异

在 Source 侧，社区已支持基于正则表达式匹配 Topic 名称的订阅方式，并与分区发现、偏移提交等机制结合^[5]。其本质是周期或事件驱动地比对“模式—当前元数据”，更新实际订阅集合。Sink 侧若要对齐类似能力，需要另行设计生产者实例生命周期与记录路由策略，不能简单照搬 Source 实现。本文当前代码中对“按模式发现 Topic”的实现落在 `DynamicKafkaSinkBuilder.setStreamPattern(...)`、`StreamPatternSubscriber` 与 `SingleClusterTopicMetadataService` 的组合上；但当模式命中新的 Topic 后，系统还要进一步将其转换为 route id、Kafka producer 配置和底层 writer，这一步正是 Sink 侧相较 Source 侧的主要额外复杂度。

为了更直观地对应本仓库中的实现逻辑，代码清单 2.1 展示了当前原型中最关键的两步：先通过 `AdminClient.listTopics()` 拉取所有 Topic，再通过正则对 `streamId` 执行匹配。这说明本文方案中的“动态发现”并不是来自 Flink 运行时的内建事件通知，而是构建在 Kafka 管理面元数据轮询之上的。

Listing 2.1 当前原型中的 Topic 元数据获取与正则匹配逻辑

```
// connector/.../SingleClusterTopicMetadataService.java
public Set<KafkaStream> getAllStreams() {
    return getAdminClient().listTopics().names().get().stream()
        .map(this::createKafkaStream)
        .collect(Collectors.toSet());
}

// connector/.../StreamPatternSubscriber.java
public Set<KafkaStream> getSubscribedStreams(KafkaMetadataService
kafkaMetadataService) {
    Set<KafkaStream> allStreams = kafkaMetadataService.getAllStreams();
    List<KafkaStream> matches = new ArrayList<>();
    for (KafkaStream kafkaStream : allStreams) {
        String streamId = kafkaStream.getStreamId();
        if (streamPattern.matcher(streamId).find()) {
            matches.add(kafkaStream);
        }
    }
    return Set.copyOf(matches);
}
```

结合代码可以看到，当前实现默认把 `streamId` 等价于 Topic 名称，因此 `stream_pattern` 实际上就是“对 Topic 名称做匹配”的正则表达式。这样做实现简单，也便于原型阶段验证功能；但如果未来扩展到“逻辑流名称”与“物理 Topic 名称”并不完全一致的场景，则需要在 `KafkaStream` 的建模上进一步引入更丰富的元数

据层次。

2.3.3 动态 Sink 需要解决连接器问题

现有 Kafka Sink 在统一 API 下仍以静态目标为主；FLIP-515^[6]等工作讨论动态 Sink 的方向，但生产可用的完整方案需处理：元数据扫描开销、写入路径与事务语义、状态快照与恢复后的对账等问题。从当前原型实现看，这些难点都已经在代码层面有所体现：例如 route 级 writer 的惰性创建已经实现，非事务路径下的退役 writer 清理也已经落地；但 EXACTLY_ONCE 路径下的安全回收、事务边界与复杂故障验证仍待补足。因此，Flink Kafka Connector 的“动态化”并不是单点功能，而是一组围绕 Writer、状态、提交与元数据服务的协同扩展。

2.4 本章小结

本章围绕后续设计所需的关键技术进行了有针对性的说明：Kafka 部分重点讨论 Topic 元数据、写入目标绑定、事务型 producer、Exactly-once 与 read_committed 可见性；Flink 部分重点说明 Checkpoint、统一 Sink API、两阶段提交和广播状态如何支撑动态连接器；Connector 部分则分析了静态 Kafka Sink 与动态 Sink 在目标发现、writer 管理和 route 级提交上的差异。这些内容直接对应第 3 章的需求边界、第 4 章的系统设计以及第 5 章的实验解释，避免把相关技术章节写成与本文工作脱节的通用背景介绍。

第三章 系统需求分析

3.1 系统应用场景分析

在实时数据平台中，Flink 作业通常承担持续运行的计算与写出职责，而 Kafka Topic 则经常随着业务边界变化而调整。例如，多租户系统可能按租户自动创建 Topic，日志平台可能按日期、地域或业务线拆分 Topic，灰度迁移和容灾切换场景中也可能同时存在新旧 Topic 或新旧集群。若仍采用传统静态 Sink，作业在提交时就把目标 Topic 与连接参数固定下来，一旦下游目标变化，就需要停止作业、修改配置并重新部署。这一过程会引入运维窗口、恢复等待时间和潜在重复写入风险，与实时链路“持续写入、低人工介入、快速接纳新目标”的要求相矛盾。

因此，本文面对的核心问题不是“如何把数据写入一个 Kafka Topic”，而是“当可写 Topic 集合在运行期间变化时，Flink Sink 如何在不重启作业的前提下发现变化、更新写入目标，并保持状态与提交路径可恢复”。这一问题天然包含两个层次：第一，系统需要从 Kafka 集群元数据中识别符合规则的新 Topic；第二，系统需要把业务侧的逻辑路由与实际物理 Topic 解耦，使业务记录不必直接绑定某个固定目标。前者对应自动发现路径，后者对应显式逻辑 route 路径。

在本项目中，connector 通过 stream_pattern、route_map、stream-metadata-discovery 等参数暴露动态能力，而应用层只负责把这些参数装配进运行中的 Flink 作业。因此，系统不仅要支持“自动发现符合模式的 Topic”，还要支持“根据业务 route id 动态决定消息应写往哪一类 Topic”。这意味着问题已从“静态写一个 Topic”扩展为“运行期维护逻辑路由、物理 Topic 与底层 KafkaWriter 的关系”。

与抽象需求相对应，当前仓库中的配置已经给出了较为完整的输入形式，本文将其整理至附录 1.1。从这些配置可以看出，该 connector 同时暴露了自动发现所需的 stream_pattern 与 discovery_interval_ms，以及显式路由所需的 route_map。这意味着本文讨论的并不是单一路径下的动态写入，而是“自动发现路径”和“逻辑 route 路径”并存的复合场景。

3.2 功能需求分析

由上述场景可知，静态 Sink 的不足并不只是一项配置不够灵活，而是会沿着“目标发现、目标选择、写入通道、状态恢复”形成连续影响：如果系统无法发现新 Topic，就不能减少停机变更；如果无法把逻辑 route 映射到物理 Topic，业务记录仍会被固定目标约束；如果发现目标后不能自动维护 writer，数据仍无法真正写出；如果 route 状态不能进入 Checkpoint，故障恢复后又会丢失运行期动态信息。因此，本节将功能需求拆分为动态 Topic 发现、自动订阅、动态路由和数据处理四类，它们分别对应前述问题链条中的不同环节，并共同支撑后续第 4 章的系统设计。

3.2.1 动态 Topic 发现需求

系统应根据用户配置的 Topic 名称模式，周期性查询 Kafka 当前 Topic 列表，并识别所有符合规则的目标。当前实现中，`DynamicKafkaSinkBuilder.setStreamPattern(...)` 接收 Pattern，`StreamPatternSubscriber.getSubscribedStreams(...)` 再调用 `KafkaMetadataService.getAllStreams()` 完成匹配；在单集群实现 `SingleClusterTopicMetadataService` 中，`streamId` 直接等价于 Topic 名称，因此模式匹配实质上就是对集群 Topic 名称做运行期过滤。

3.2.2 自动订阅需求

Sink 侧的“订阅”不是消费者组意义上的分区订阅，而是指 Writer 对“当前可写目标集合”的维护。在代码中，这一集合由 `DynamicKafkaSinkWriter` 的 `activeRouteIds` 表示；当 `refreshRouteIfNeeded(...)` 触发刷新时，Writer 通过 `resolveRoutes()` 获取新的可写 route 集合，再按 route id 检查是否需要创建新的 `ClusterWriter`。因此，本系统必须支持以下功能：一是首次启动时根据元数据建立可写集合；二是在运行期发现新增 route 时自动扩展写入路径；三是在 route 不再被活动集合或显式逻辑 route 引用时，尽可能释放对应 writer，避免长时间累积陈旧资源。

需要指出的是，当前代码已经支持 route 的新增与切换，并在 `DeliveryGuarantee.NONE` 和 `AT_LEAST_ONCE` 路径下补充了退役 writer 清理；但在 `EXACTLY_ONCE` 路径下，由于事务上下文可能跨 Checkpoint 存在，当前实现仍采用保守策略，不自动清理退役 writer。因此，自动订阅需求在本文原型中已经形成基本

闭环，但事务路径下的安全回收仍属于后续工程化完善方向。

3.2.3 动态路由需求

系统应支持“逻辑 route id → 物理 Kafka 目标”的两级映射，而不是把业务记录直接绑定到固定 Topic。在应用层，DynamicSinkEvent 同时实现了数据记录与路由更新事件两种语义：普通数据事件携带 routeId 与消息内容，更新事件通过 getRouteUpdates() 下发 Map<String, KafkaRouteDestination>。在运行时，DynamicKafkaSinkWriter.write(...) 会先判断输入事件是否为 DynamicKafkaRouteEvent，若是更新事件则调用 applyRouteUpdates(...) 更新 explicitRoutesById；若是普通事件且显式指定了 routeId，则调用 writeToExplicitRoute(...) 走逻辑路由路径。

这说明本系统的路由需求具有两个层面：其一，面向“自动发现”的通用 route；其二，面向“显式 route id”的逻辑路由。后者正是本文“逻辑路由与物理路由分离”设计思想的直接代码体现。

进一步地，当前原型并没有把 route 配置静态地塞进 Sink，而是通过一个轻量级验证载体利用广播状态与控制事件把 route 更新传播到运行中的算子实例。代码清单 3.1 展示了这一机制的关键处理逻辑：普通数据事件只有在广播状态中存在对应 routeId 时才会被放行，而 route 更新事件则会先改写广播状态，再继续向下游 Sink 传播。需要强调的是，这一部分并非 connector 的核心交付物，而是为了验证 connector 的显式 route 更新接口而构造的上层驱动。

Listing 3.1 应用层通过广播状态维护 route 配置并向下游传播控制事件

```
@Override
public void processElement(
    DynamicSinkEvent value, ReadOnlyContext ctx, Collector<
    DynamicSinkEvent> out)
    throws Exception {
    if (value == null || value.isRouteUpdate() || value.getRouteId() == null)
    {
        return;
    }
    if (ctx.getBroadcastState(routeMapStateDesc).contains(value.getRouteId())
    ) {
        out.collect(value);
    }
}

@Override
public void processBroadcastElement(
```

```

        DynamicSinkEvent value, Context ctx, Collector<DynamicSinkEvent> out
    )
    {
        throws Exception {
        if (value == null || !value.isRouteUpdate()) {
            return;
        }
        for (Map.Entry<String, KafkaRouteDestination> entry : value.
            getRouteUpdates().entrySet()) {
            if (entry.getValue() == null) {
                ctx.getBroadcastState(routeMapStateDesc).remove(entry.getKey());
            } else {
                ctx.getBroadcastState(routeMapStateDesc).put(entry.getKey(),
                    entry.getValue());
            }
        }
        out.collect(value);
    }
}

```

这一实现意味着系统的功能需求实际上还隐含了一个重要约束：控制面配置必须先于数据面生效，否则普通数据即使携带了合法的逻辑 **route id**，也会因为广播状态中尚不存在该 **route** 而被过滤掉。因此，本文在分析动态路由需求时，需要同时考虑“路由如何表示”和“路由如何传播”两个问题。

3.2.4 数据处理需求

系统应能在持续数据流上稳定完成以下处理链路：数据生成/采集、**route** 校验、路由解析、Topic 选择、消息序列化、Kafka 写入、Checkpoint 快照与故障恢复。当前验证载体使用 `DataGeneratorSource` 生成测试数据，并通过 `BroadcastProcessFunction` 先过滤掉不存在的 **route id**，再将合规事件送入动态 Sink。说明系统在验证层面已经具备“数据面”和“控制面”混合流的基本处理能力，而 **connector** 本身则只关心进入 **Writer** 之后的解析、写入与状态管理。

从实现要求看，序列化层还必须适应动态 **Topic** 绑定。当前 `DynamicKafkaSinkWriter.createClusterWriter(...)` 会为每个 **route** 克隆一份 `KafkaRecordSerializationSchema`，并用内部类 `FixedTopicKafkaRecordSerializationSchema` 覆写最终发送的 **Topic**。若用户 **serializer** 实现了 `DynamicKafkaTargetAwareRecordSerializationSchema`，则可以直接拿到解析后的目标 **Topic**；否则框架会保留原有 **key/value/headers**，只强制改写 **Topic** 字段。这一机制是当前原型可运行的关键约束之一。

3.3 非功能需求分析

为了避免非功能需求停留在定性描述，本文将其限定为当前原型能够在本地实验中验证的指标。这些指标不是生产级服务等级协议，而是用于判断本文方案是否满足毕业设计原型目标的可测约束。各项指标的取值依据来自两类因素：其一是系统配置的理论上限，例如 `discovery_interval_ms=2000` 决定了新 Topic 最坏情况下至少要等待下一轮刷新；其二是本地实验的合理可观测范围，例如第 5 章采用 `parallelism=1`、小规模 route 数量和约 30 秒运行窗口进行验证。表 3-1 给出了本文采用的非功能需求指标及其后续验证位置。

表 3-1 本文原型的非功能需求指标

需求类型	量化指标	指标依据与验证方式
发现实时性	在 <code>discovery_interval_ms=2000</code> 条件下，新增匹配 Topic 的首次写入延迟应不超过 3 s。	理论上延迟主要由下一轮刷新等待时间、 <code>AdminClient.listTopics()</code> 调用和任务调度组成；第 5 章通过发现实时性实验验证。
吞吐可接受性	在本地单机、 <code>AT_LEAST_ONCE</code> 、2-route 显式路由场景下，动态方案吞吐较静态基线下降不超过 5%。	动态 Sink 增加 route 解析和 writer 查找，但不应在小规模 route 下成为主导瓶颈；第 5 章静态基线对照实验验证。
小规模扩展性	在相同本地环境下，route 数量由 2 增至 4 时，吞吐不应出现超过 10% 的下降。	该指标用于验证小规模多 route 管理不会立刻压垮写入路径；第 5 章吞吐测试表进行验证。
Checkpoint 协同	打开 <code>checkpoint-interval-ms=5000</code> 后，作业应能连续完成 Checkpoint，稳态完成时延控制在 100 ms 以内。	小规模作业；第 5 章 Checkpoint 时延实验验证。
事务路径可运行性	<code>EXACTLY_ONCE</code> 小规模运行中应完成至少 5 次 Checkpoint，并在两个目标 Topic 均产生 committed 输出。	该指标验证 route 级事务提交链路可运行，但不等同于复杂故障注入下的完整精准一次证明；第 5 章 Exactly-once 验证对应。

3.3.1 实时性要求

实时性的核心不是单条消息发送延迟，而是“Topic 变化被发现并投入写入”的时间上限。当前代码通过配置项 `stream-metadata-discovery-interval-ms` 控制元数据刷新周期；在 `DynamicKafkaSinkOptions` 中其默认值为 -1，表示非正值时关闭自动发现，而在应用层 `KafkaSinkBuilder` 中会把 `dynamic.discovery_interval_m`

传入该参数，示例配置默认是 2000 ms。因此，系统的最小发现粒度大致受该周期限制，再叠加 `AdminClient.listTopics()` 返回时间与作业线程调度延迟，形成端到端发现时延。基于这一机制，本文将本地原型的实时性目标定义为：默认 2000 ms 刷新周期下，新增匹配 Topic 从创建成功到首次出现写入的时间不超过 3 s。该阈值比配置周期多预留约 1 s 给 Kafka 管理面调用与本地任务调度，既符合当前轮询实现的理论上界，也能在第 5 章通过 offset 观测直接验证。

3.3.2 可扩展性要求

可扩展性主要体现在三个方面。首先，随着匹配 Topic 数量增加，`cluster WritersByRouteId` 中维护的底层 `KafkaSink writer` 数量同步增加，Kafka 连接、事务上下文和缓存占用也会提高。其次，`resolveRoutes()` 会对所有已订阅 stream 和其 cluster metadata 做展开，因此当 Topic 与 cluster 数量扩大时，刷新计算本身会带来额外成本。再次，route id 的设计目前采用 `clusterId|topic|bootstrapServers` 或 `logicalRouteId|clusterId|topic|bootstrapServers` 形式拼接，虽然实现简单、可唯一标识物理目标，但在大规模场景下需要更系统的命名与管理策略。

考虑到本文实验环境是本地单机原型，而非生产集群压测，本文把可扩展性指标限定在小规模 route 扩展是否引入明显退化：一是在 2-route 显式路由场景下，动态方案较静态基线吞吐下降不超过 5%；二是在相同条件下将 route 数量从 2 增加到 4 时，吞吐下降不超过 10%。这两个阈值的合理性在于：动态 Sink 相比静态 Sink 必然增加 route 查找、writer 管理和提交包装开销，但在毕业设计原型所覆盖的小规模场景中，这些开销不应主导整体吞吐；若下降超过上述范围，则说明动态管理逻辑已经成为主要瓶颈，需要在设计阶段重新考虑缓存、批量刷新或 writer 复用策略。

3.3.3 稳定性与容错性要求

动态写入系统不能只关注“能否写出去”，还必须保证故障后状态可恢复。当前实现中，`DynamicKafkaSink` 继承 `TwoPhaseCommittingStatefulSink`，`Writer` 在 `snapshotState(...)` 中输出 `DynamicKafkaSinkWriterState`，其中包含 `routeId`、`kafkaClusterId`、`topic`、`transactionalIdPrefix`、`kafkaProducerConfig` 以及底层 `KafkaWriterState` 列表；提交阶段则通过 `prepareCommit()` 生成 `DynamicKafkaCommittable`，交由 `DynamicKafkaCommitter` 按 route 分发给底层 `KafkaCommitter`。因此，系统至少需要满足：快

照内容足以重建 route 级 writer，恢复后能够重新拉起 writer 并继续工作，提交路径能够按 route 维度保持隔离。

本文对稳定性与容错性的原型级量化要求包括两项。第一，在开启 checkpoint-interval-m 后，作业应能够连续完成 Checkpoint，且排除首次冷启动影响后的稳态 Checkpoint 完成时延应控制在 100 ms 以内；该指标用于判断 route 级状态封装是否给本地小规模作业引入明显阻塞。第二，在 EXACTLY_ONCE 小规模验证中，系统应至少完成 5 次 Checkpoint，并在两个目标 Topic 上观察到 committed 输出；该指标用于证明 route 级事务提交路径能够跑通。需要强调的是，后者并不等同于覆盖网络中断、TaskManager 崩溃、事务超时等复杂故障的完整精准一次证明，而是本文原型阶段可验证的最低一致性闭环。与此同时，日志、监控指标、异常处理与 EXACTLY_ONCE 路径下退役 writer 的安全清理等工程能力，仍是进一步增强稳定性的重点。

3.4 问题建模与分析

设运行时 Kafka Topic 集合为 $T(t)$ ，由正则模式 P 过滤出的自动发现集合为 $S(t) = \{\tau \in T(t) \mid \tau \text{ 匹配 } P\}$ 。对于显式路由模式，系统还维护逻辑 route 集合 $R = \{r_1, r_2, \dots, r_n\}$ 以及映射函数 $g : R \rightarrow 2^{S(t)}$ ，表示一个逻辑 route id 在时刻 t 可以映射到的物理 Topic 集合。对任意输入记录 e ，若其未显式指定 route id，则走自动发现路径并广播写入当前 activeRouteIds；若其显式指定逻辑 route id，则写入集合由 $g(r, t)$ 决定。

当前原型的一个重要实现特征是“自动发现路径”和“显式逻辑路由路径”并存：前者由 resolveRoutes() 基于订阅的 KafkaStream 自动展开，后者由 resolveExplicitRoutes(...) 基于 KafkaRouteDestination 的 bootstrap Servers 与 topicPattern 解析得到。因此，系统本质上是在同一 Writer 内维护两类 route 解析逻辑，并保证两者都能落入统一的快照与提交框架中。

3.5 本章小结

本章从实时链路长期运行与 Kafka Topic 动态变化之间的矛盾出发，独立推导了本文系统需要解决的问题：既要在运行期发现符合规则的新 Topic，也要将业务逻辑 route 与物理 Topic 解耦，并让 writer、状态和提交路径跟随 route 变化而调整。随后，本章将这些问题拆解为动态 Topic 发现、自动订阅、动态路由和数据处理四类功能需

求, 并进一步给出发现实时性、吞吐可接受性、小规模扩展性、Checkpoint 协同和事务路径可运行性等量化非功能指标。这些指标将在第 4 章的设计中分别对应到元数据刷新、route 解析、writer 管理、状态快照和提交器设计, 并在第 5 章通过发现实时性、吞吐可接受性、小规模扩展性、Checkpoint 协同与事务路径可运行性实验进行验证, 从而形成需求、设计与实验之间的闭环。

第四章 基于 Flink 的动态 Topic 订阅系统设计与实现

4.1 系统总体架构设计



第 3 章已经将系统需求归纳为四类功能需求和若干可验证的非功能指标。由这些需求出发,可以发现动态 **Kafka Sink** 的设计不能简单地在原有 **KafkaSink** 外层增加一个正则判断:若只解决 **Topic** 发现,数据仍不知道应该写向哪个 **writer**;若只解决路由映射,新增 **Topic** 仍无法自动进入可写集合;若只在应用层手动创建 **producer**,又会脱离 **Flink Checkpoint** 和两阶段提交机制。因此,本章的设计目标是在尽量复用现有 **Kafka Sink** 写入能力的前提下,增加一层能够感知元数据、维护 **route**、管理 **writer** 生命周期并参与状态提交的动态控制层。

围绕这一目标,本文采用“动态外壳 + 静态内核”的总体架构。所谓静态内核,是指每个已经解析出的物理 **route** 仍复用普通 **KafkaSink writer** 完成序列化、发送、刷写和事务提交;所谓动态外壳,则由 **DynamicKafkaSinkWriter** 负责在运行期把 **Topic** 元数据、逻辑 **route** 和底层 **writer** 连接起来。这一选择对应第三章的非功能约束:复用底层 **KafkaSink** 可降低小规模动态路由相对静态基线的吞吐损失;接入统一 **Sink API** 可保证 **Checkpoint** 与 **EXACTLY_ONCE** 路径仍有可验证的提交链路;通过周期性元数据刷新则可以用 **discovery_interval_ms** 明确控制新增 **Topic** 的发现时延。

具体到连接器内部,本系统划分为动态 **Sink** 构建层、运行时写入层、元数据发现层以及状态与提交层四个核心层次。这种划分不是单纯按类名归类,而是对应一条从需求到实现的推导链:构建层负责把用户配置转化为可执行的订阅器和元数据服务;元数据发现层负责解决“有哪些 **Topic** 可写”;运行时写入层负责解决“当前记录写到哪些 **route** 以及如何复用 **writer**”;状态与提交层负责解决“故障恢复后这些动态 **route** 如何继续提交”。应用侧作业只作为验证载体,用于向 **connector** 提供测试数据与控制事件,而不承担核心动态写入语义。

其中,动态 **Sink** 构建层由 **DynamicKafkaSink.builder()** 与 **DynamicKafkaSinkBuilder** 组成,负责选择订阅模式、绑定元数据服务、设置序列化器与投递语义;运行时写入层由 **DynamicKafkaSinkWriter** 主导,承担 **route** 刷新、**route** 解析、多 **writer** 写入、快照与提交前准备等核心逻辑;元数据发现层由 **SingleClusterTopicMetadataService** 通过 **AdminClient** 访问 **Kafka** 集群

并返回 `KafkaStream / ClusterMetadata` 结构；状态与提交层则由 `DynamicKafkaSinkWriterState`、`DynamicKafkaCommittable` 和 `DynamicKafkaCommitter` 共同组成，负责 **route** 级状态恢复与 **route** 级提交转发。

从连接器类之间的调用关系看，系统整体链路可以概括为：上层作业把数据流交给 `DynamicKafkaSink`；`DynamicKafkaSink` 在 `createWriter(...)` 或 `restoreWriter(...)` 中创建 `DynamicKafkaSinkWriter`；**Writer** 通过 `KafkaStreamSubscriber` 与 `KafkaMetadataService` 解析当前有效 **route**，并按 **route** 懒加载底层 `KafkaSink writer`；**Checkpoint** 到来时，**Writer** 把每个 **route** 对应的底层 `KafkaWriterState` 包装为 `DynamicKafkaSinkWriterState`；提交时，则把底层 `KafkaCommittable` 封装为 `DynamicKafkaCommittable`，再由 `DynamicKafkaCommitter` 按 **route fan-out** 给真实的 `KafkaCommitter`。

如果从“上层如何调用 **connector**”这一角度观察，应用层只需完成三件事：准备数据流、准备控制事件流、并在构建阶段把 `streamPattern`、元数据服务和 `serializer` 注入 `DynamicKafkaSink`。因此，论文的重点不在于作业装配本身，而在于这些参数进入 **connector** 后如何驱动后续的 **route** 解析、**writer** 管理和状态提交。具体配置样例与运行命令已整理到附录 1.1 与附录 1.2。

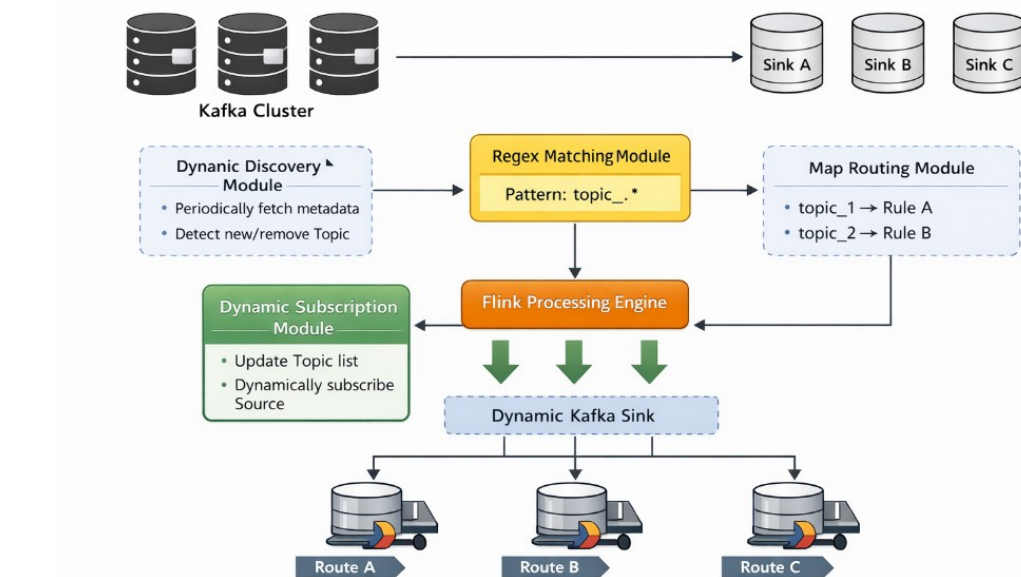


图 4-1 动态 Kafka Sink 总体架构示意（区分数据流、控制流与依赖关系）

图 4-1 对应本文方案在概念层面的总体结构：Kafka 集群元数据先进入动态发现模块，随后由正则匹配模块筛选目标，再与映射路由模块共同决定写入路径，最终由 Flink 侧的动态 Sink 完成多目标输出。为避免把运行期数据流与静态依赖关系混淆，本文在阅读该类架构图时统一采用如下约定：业务数据与 route 更新事件属于运行期流动关系，表示“记录或控制事件如何进入 Sink 并触发写入”；构建器、元数据服务、订阅器与底层 KafkaSink 之间属于依赖或组合关系，表示“某个组件持有或调用另一个组件”；Checkpoint state 与 committable 则属于状态/提交关系，表示“运行期结果如何进入 Flink 容错与提交链路”。后续图 4-2 也遵循这一阅读口径。

4.2 系统模块划分

本节进一步说明各模块的划分依据。第三章中的问题链条可以概括为“先知道有哪些目标，再决定写向哪些目标，随后维护实际写入通道，最后把通道状态纳入恢复与提交”。因此，模块划分也遵循这一顺序，避免把所有动态逻辑集中在一个巨大的 Writer 方法中。表 4-1 总结了主要需求与设计选择之间的对应关系。

表 4-1 需求到设计选择的对应关系

需求或约束	设计选择	设计动机
新 Topic 在秒级被发现	AdminClient 周期轮询 + stream-metadata-discovery	Kafka 缺少适合 Sink 端直接订阅的 Topic 变更事件，周期轮询实现简单且发现上界可由配置控制。
小规模动态路由吞吐接近静态基线	route 解析缓存 + 底层 Kafka Sink writer 懒创建复用	避免每条记录都重新创建 producer，同时复用成熟写入路径以减少额外开销。
逻辑 route 可运行期更新	广播状态传递控制事件 + Sink 内部维护 explicitRoutes ById	广播状态保证并行实例收到一致配置，Sink 内部表保证控制事件能直接影响后续写入。
Checkpoint 协同与事务路径可运行性	route 级 state、committable 和 committer	动态 route 不能脱离 Flink 容错体系，提交对象必须携带 route 元数据才能分组恢复和提交。
route 变化下资源可控	非事务路径清理退役 writer，事务路径保守保留	在资源回收和事务安全之间取折中，避免 EXACTLY_ONCE 下过早关闭仍可能参与提交的 writer。

4.2.1 Topic 动态发现模块

动态发现模块对应 `SingleClusterTopicMetadataService` 与 `StreamPatternSubscriber` 的组合。`SingleClusterTopicMetadataService` 在 `getAllStreams()` 中调用 `AdminClient.listTopics().names().get()`, 把每个 **Topic** 封装为一个 `KafkaStream`, 并进一步放入以 `clusterId` 为键的 `ClusterMetadata` 中; 在该实现里, 一个 **stream** 实际上就等价于一个 **Topic**, 因此 **stream** 发现就是 **Topic** 发现。`StreamPatternSubscriber` 则在 `getSubscribedStreams(...)` 中遍历所有 **stream**, 对 `streamId` 应用正则匹配, 并返回命中的集合。

这里的关键设计决策是采用 **Kafka** `AdminClient` 主动查询, 而不是在 **Flink** 作业外额外引入配置中心或服务注册组件。原因在于本文的需求目标是验证 **Kafka Topic** 自身变化能否被 **Sink** 感知, 而 `AdminClient.listTopics()` 正好提供了与 **Kafka** 集群真实状态直接对齐的管理面入口; 同时, 配合第三章定义的 3 s 发现目标, 轮询周期可以通过 `stream-metadata-discovery-interval-ms` 显式控制。其局限也很明确: 当前实现默认单集群, 且每次刷新都需要重新遍历可见 **Topic** 列表, 尚未引入增量缓存或事件通知机制。

4.2.2 正则匹配模块

正则匹配在当前实现中出现于两个位置。其一是构建阶段的 `stream pattern`: `DynamicKafkaSinkBuilder.setStreamPattern(Pattern streamPattern)` 创建 `StreamPatternSubscriber`, 用于自动发现路径。其二是显式 `route` 路径中的 `KafkaRouteDestination.topicPattern`: `DynamicKafkaSinkWriter.resolveExplicitRoutes(...)` 会再次编译该正则, 并同时对其 `stream.getId()` 和候选 `topic` 执行匹配。由于在 `SingleClusterTopicMetadataService` 中 `streamId` 就是 `topic` 名称, 这种“双重匹配”在当前单集群实现里略显重复, 但它为未来扩展“逻辑 **stream** 与物理 **topic** 不完全等价”的实现留下了接口空间。

选择正则表达式作为匹配方式, 是因为第三章中的多租户、日期滚动和业务线拆分场景都具有明显的命名规则。相比维护完整 **Topic** 白名单, 正则配置可以用较短规则覆盖一组动态增长的 **Topic**; 相比把规则写入代码, 正则又能通过配置传入, 避免每次新增目标都重新部署作业。为了控制误命中风险, 本文在后续实验配置中采用锚

定表达式, 例如 `^exp-auto-.*$` 或 `^exp-dynamic-c$`, 使匹配范围和预期目标保持一致。

4.2.3 自动订阅模块

自动订阅模块的核心入口是 `DynamicKafkaSinkWriter.refreshRouteIfNeeded(...)`。该方法维护两个关键变量: `activeRouteIds` 表示当前生效的自动发现 route 集合, `nextMetadataRefreshTimestamp` 表示下一次允许执行元数据刷新的时间戳。当刷新条件满足时, **Writer** 调用 `resolveRoutes()` 重新计算 route 列表, 再通过 `clusterWritersByRouteId.computeIfAbsent(...)` 为尚不存在的 route 创建新的 `ClusterWriter`。

在本文当前版本的实现中, route 生命周期管理已经比前一阶段更进一步: 对于 `DeliveryGuarantee.NONE` 与 `AT_LEAST_ONCE` 路径, **Writer** 会在刷新、刷写、提交准备与状态快照之后调用 `cleanupRetiredWriters()`, 关闭那些既不属于当前 `activeRouteIds`, 也不再被显式逻辑 route 引用的历史 writer, 从而避免其长期留在内存与提交链路中。不过, 对于 `EXACTLY_ONCE` 路径, 当前实现仍采取保守策略, 暂不自动清理退役 writer, 以避免过早关闭事务上下文带来状态不一致问题。因此, 本系统已经初步具备了 route 生命周期闭环, 但仍保留了“事务路径下安全回收”这一后续优化空间。

这一策略体现了实时性与资源治理之间的折中: 为了满足新增 Topic 秒级可写的需求, 刷新时应尽快把新 route 转换为可用 writer; 为了满足小规模吞吐指标, 又不能在每条记录到来时都重新创建 writer。因此, 系统采用“全量重算活动集合 + 惰性创建缺失 writer + 条件清理退役 writer”的方式。全量重算牺牲了一部分大规模场景下的刷新效率, 但降低了状态差分维护复杂度, 适合本文当前的原型验证范围。

4.2.4 动态路由模块

动态路由模块由 `DynamicKafkaRouteEvent`、`DynamicSinkEvent`、`KafkaRouteDestination` 以及 `DynamicKafkaSinkWriter` 中的 `explicitRoutesById`、`explicitResolvedRouteIdsByLogicalId` 两张表共同构成。`DynamicSinkEvent` 既可表示普通数据事件, 也可表示路由更新事件; `KafkaRouteDestination` 封装 `kafkaClusterId`、`bootstrapServers` 与 `topicPattern` 三元组, 是逻辑 route id 对应的物理目标描述。

在验证载体中, `DynamicJob` 使用 `MapStateDescriptor<String, Kafka`

RouteDestination> 建立广播状态, 将 YAML 中的 route_map 读入后封装为一次 DynamicSinkEvent.update(...) 广播事件, 再由 BroadcastProcessFunction 写入广播状态并原样输出到下游 Sink。这样, Sink 侧既能收到业务数据, 也能收到 route 更新控制消息, 形成“控制面与数据面共流”的运行模型。

采用广播状态而不是让每个 Sink 子任务独立读取配置文件, 是为了保证并行实例看到一致的 route 更新。如果每个子任务各自读取本地文件或远程配置, 不同实例可能在不同时间观察到不同 route 版本, 从而使同一批数据被写入不一致的目标。广播状态由 Flink 运行时负责分发到所有并行实例, 更符合第三章中“控制面配置必须先于数据面生效”的约束; Sink 内部再通过 DynamicKafkaRouteEvent 识别控制事件, 使配置变化可以沿着同一条数据流进入 writer。

需要指出的是, 当前版本还补充了逻辑 route 的删除语义: 在 route 更新事件中, 若某个 routeId 对应的 KafkaRouteDestination 为 null, 则表示该逻辑 route 被显式移除。应用层广播状态会执行 remove(routeId), Sink 侧则同步从 explicitRoutesById 与 explicitResolvedRouteIdsByLogicalId 中删除对应项。这意味着 route 现已不再只有“新增和覆盖”, 而是具备了基本的“新增、更新、删除”三种管理语义。

4.2.5 动态 Sink 模块

DynamicKafkaSink 是整个动态写入机制的统一入口。它实现了 TwoPhaseCommittingStatefulSink<IN, DynamicKafkaSinkWriterState, DynamicKafkaCommittable>, 说明它同时支持 writer 状态恢复与提交器路径。类内部保存 KafkaStreamSubscriber、KafkaMetadataService、KafkaRecordSerializationSchema、DeliveryGuarantee、transactionalIdPrefix 与 metadataRefreshIntervalMs 等关键配置, 并在 createWriter(...) 与 restoreWriter(...) 中把这些配置传给 DynamicKafkaSinkWriter。

与静态 KafkaSink 不同, 动态 Sink 本身不直接向某一个固定 Topic 发送数据, 而是把“写到哪”推迟到 Writer 运行时决定。因此, DynamicKafkaSink 的主要职责不是发送消息, 而是为 Writer 提供正确的订阅器、元数据服务、公共 Producer 属性与提交语义。

4.3 动态 Topic 发现机制设计

4.3.1 Kafka 元数据获取方式

动态发现机制的首要问题是“从哪里获得可信的 Topic 集合”。一种选择是让业务配置显式下发所有 Topic，但这会把动态发现退化为人工维护列表；另一种选择是引入外部注册中心，但会增加系统依赖并使验证重点偏离 Kafka 本身。因此，本文选择直接从 Kafka 管理面获取元数据。当前实现中的元数据获取采用主动查询方式。`SingleClusterTopicMetadataService.getAdminClient()` 会延迟初始化一个 `Kafka AdminClient`，并为其补充 `client.id`，然后在 `getAllStreams()` 中调用 `listTopics()` 返回当前所有 Topic 名称；每个 Topic 会被封装为一个 `Kafka Stream(topic, clusterMetadataMap)`。这种设计的优点是实现简单、与 Kafka 官方管理接口一致；缺点是刷新频率高时会增加 `AdminClient` 调用压力。

代码清单 4.1 给出了该过程的关键实现。可以看到，`SingleClusterTopicMetadataService` 实际上把“Topic 列表”直接提升为了 `Set<KafkaStream>`，因此后续订阅器与 `Writer` 可以统一围绕 `KafkaStream` 抽象工作，而不必直接依赖 `AdminClient` 的返回结构。

Listing 4.1 单集群元数据服务通过 AdminClient 拉取全部 Topic

```
@Override
public Set<KafkaStream> getAllStreams() {
    try {
        return getAdminClient().listTopics().names().get().stream()
            .map(this::createKafkaStream)
            .collect(Collectors.toSet());
    } catch (InterruptedException | ExecutionException e) {
        throw new RuntimeException("Fetching all streams failed", e);
    }
}
```

由于 `ClusterMetadata` 中同时存储 topics 与 properties，动态 Sink 在后续创建 route writer 时，不仅能拿到目标 Topic，也能继承该 cluster 对应的连接参数。这为未来多集群扩展提供了基础。

4.3.2 Topic 变化检测策略

与许多显式维护“上一轮 Topic 集合差分”的实现不同，当前 `Writer` 采用更直接的刷新策略：每次 `refreshRouteIfNeeded(...)` 触发时重新调用 `resolveRoutes()`，把当前元数据投影为新的 route 集合，再用 `activeRouteIds =`

`routes.stream().map(...).collect(...)` 覆盖活动集合。也就是说，当前代码层面并未单独维护“新增 / 删除 / 保持不变”的差分表，而是通过“重算活动 route 集合 + 按需创建新 writer”的方式实现动态扩写。

这种实现降低了状态管理复杂度，但也带来两个后果：一是 route 删除后旧 writer 仍可能保留在 `clusterWritersByRouteId` 中；二是当 Topic 数量很多时，全量重算的刷新成本会高于增量差分。在当前版本中，前一个问题已经在非事务路径上得到部分缓解，因为退役 writer 会在合适时机被关闭并移除；但对于事务路径和大规模场景，全量重算与保守回收策略仍然意味着较高的工程复杂度。

之所以没有在当前阶段引入复杂差分表，是因为第三章定义的实验范围主要关注 2 到 4 个 route 的小规模原型验证。在这一范围内，差分算法带来的收益有限，却会增加恢复时需要保存和对齐的状态。相比之下，全量重算只要求 `KafkaMetadataService` 返回当前真实集合，Writer 再根据 route id 判断是否需要创建或清理 writer，更容易保证需求、设计和实验之间的闭环。

4.3.3 自动订阅列表更新机制

自动发现路径的 route 解析由 `resolveRoutes()` 完成：先调用 `kafkaStreamSubscriber.getSubscribedStreams(...)` 得到订阅 stream 集合，再遍历每个 stream 的 `clusterMetadataMap`，筛选 `isClusterActive(...)` 为真的 cluster，最终为每个 `(clusterId, topic, bootstrapServers)` 组合构造一个 `ResolvedRoute`。`ResolvedRoute` 内部包含 `routeId`、`kafkaClusterId`、`topic`、`transactionalIdPrefix` 与 `kafkaProducerConfig`，它是底层 writer 创建的直接输入。

route id 的构造方式也值得说明：自动发现路径使用 `clusterId|topic|bootstrapServers`，显式 route 路径使用 `logicalRouteId|clusterId|topic|bootstrapServers`。这种 route id 设计既保证了物理写入目标的唯一性，也方便在提交与恢复阶段按 route 维度分组。

4.4 基于正则表达式的匹配机制

4.4.1 Pattern 设计原则

正则匹配机制的设计目标是把“动态增长的 Topic 集合”转换为“可被配置规则稳定描述的目标集合”。从代码实现看，Pattern 的使用遵循两个原则。第一，构

建期即完成编译，避免在主处理路径重复创建 `Pattern` 对象；第二，`Pattern` 应尽量与 `Topic` 命名规范一致，避免过宽正则带来大范围误命中。例如，配置中的 `^dynamic-kafka-sink-a$` 与 `^dynamic-kafka-sink-b$` 就显式限定了完整 `Topic` 名称，比简单的模糊包含匹配更安全。

此外，显式 `route` 的 `topicPattern` 不仅决定了将来能命中的 `Topic`，也隐含决定了 `route` 的 `fan-out` 范围。若一个正则可能匹配多个 `Topic`，则一个逻辑 `route id` 可能解析成多个物理 `route`。这种行为是当前设计允许的，但也会带来更高的 `writer` 数量与更复杂的提交路径。

4.4.2 匹配流程与实现

自动发现路径中，`StreamPatternSubscriber` 对 `streamId` 逐一执行 `matcher(...).find()`；显式路由路径中，`resolveExplicitRoutes(...)` 首先根据 `KafkaRouteDestination.topicPattern` 编译正则，然后遍历 `kafka MetadataService.getAllStreams()` 返回的所有 `stream`。若 `stream` 命中模式，再继续遍历其 `clusterMetadataMap` 与具体 `topic` 集合，对 `topic` 再做一次正则匹配。匹配成功后，`Writer` 用逻辑 `route id`、`clusterId`、`topic` 与 `bootstrapServers` 组装新的物理 `route id`，并调用重载的 `createResolvedRoute(...)` 生成 `route` 描述。

这一流程说明：当前系统的逻辑 `route` 不是直接绑定“某一个固定 `topic`”，而是绑定“一个可解析的物理目标集合”。这正是本文“逻辑路由与物理路由分离”设计思想在代码层面的实现落点。

4.5 自动订阅机制实现

4.5.1 动态写入目标更新策略

自动订阅机制需要同时处理两类输入：一类是普通业务数据，另一类是 `route` 更新控制事件。如果二者分别走完全独立的通道，系统需要额外处理控制通道与数据通道的同步问题；本文选择让控制事件也进入 `Sink`，由 `Writer` 在写入入口处分流，从而把配置变化与后续数据写入纳入同一执行顺序。`DynamicKafkaSink Writer.write(...)` 是运行时主入口。该方法首先判断输入元素是否实现了 `DynamicKafkaRouteEvent`。若是 `route` 更新事件，则直接调用 `applyRoute Updates(...)` 更新逻辑路由表；若元素显式携带 `routeId`，则进入 `write ToExplicitRoute(...)`；否则进入自动发现路径，先调用 `refreshRouteIf`

Needed(false)，再把当前记录写到所有 activeRouteIds 对应的 writer 中。

这一路径在代码层面非常清晰，如代码清单 4.2 所示。该实现有两个值得注意的特点：其一，控制事件和业务数据共用同一输入流，但在 Writer 内部被快速分流；其二，自动发现路径默认把普通记录复制到全部活动 route，而不是只选取其中一个目标。

Listing 4.2 DynamicKafkaSinkWriter 的主写入路径

```
@Override
public void write(IN element, Context context)
    throws IOException, InterruptedException {
    if (element instanceof DynamicKafkaRouteEvent) {
        DynamicKafkaRouteEvent routeEvent =
            (DynamicKafkaRouteEvent) element;
        if (routeEvent.isRouteUpdate()) {
            applyRouteUpdates(routeEvent.getRouteUpdates());
            return;
        }
        String routeId = routeEvent.getRouteId();
        if (routeId != null) {
            writeToExplicitRoute(routeId, element, context);
            return;
        }
    }
    refreshRouteIfNeeded(false);
    Preconditions.checkNotNull(!activeRouteIds.isEmpty(),
        "No active routes resolved for DynamicKafkaSink.");
    for (String routeId : activeRouteIds) {
        ClusterWriter<IN> clusterWriter = clusterWritersByRouteId.get(
            routeId);
        Preconditions.checkNotNull(clusterWriter)
            .writer.write(element, context);
    }
}
```

writeToExplicitRoute(...) 的实现很能体现系统特点：若逻辑 route id 首次出现，Writer 会先在 explicitResolvedRouteIdsByLogicalId 中检查是否已解析过物理 route；若没有，则调用 resolveExplicitRoutes(...) 根据 Kafka RouteDestination 解析出物理 route 列表，并为每个物理 route 创建底层 writer。随后，同一逻辑 route id 的后续消息即可直接复用这些物理 writer。

这说明当前系统不是简单地“每条消息临时查一次 Topic 并发送”，而是把 route 解析结果缓存下来，以减少重复创建 writer 的开销。

4.5.2 无重启更新机制设计

无重启更新体现在应用层与 Sink 层的协同。应用层 DynamicJob 使用 BroadcastStream 把 route 更新事件发送到所有并行实例；RouteBroadcastProcessFunction.processBroadcastElement(...) 将其写入广播状态，同时继续向下游输出该事件。Sink 接收到该事件后，DynamicKafkaSinkWriter 在 applyRouteUpdates(...) 中直接更新本地 explicitRoutesById，并清除旧的 explicitResolvedRouteIdsByLogicalId 缓存，使下一条数据到来时触发重新解析。

这种做法的优点是实现简单，route 更新不需要停止作业；缺点是 route 更新事件与普通数据事件共流，因此要依赖输入顺序保证“先更新、后使用”的基本时序。当前 BroadcastProcessFunction 已在进入 Sink 前做了一层 route 过滤，但在更复杂场景下，仍可进一步研究显式版本号、双缓冲配置或 barrier 对齐等方案。相比前一阶段，本版本已经把“route 删除”纳入无重启更新语义中，使逻辑配置的热更新更加完整。

这一设计直接服务于第三章的“无需重启”和“控制面配置先于数据面生效”需求。广播状态解决配置分发一致性，applyRouteUpdates(...) 解决 Sink 内部路由表更新，解析缓存失效则保证下一条数据不会继续使用旧目标。三者组合起来，形成了从配置更新到写入目标切换的完整链路。

4.6 动态路由机制设计（重点）

4.6.1 Topic 与业务逻辑映射模型

第三章指出，业务记录不应直接绑定物理 Topic，否则 Topic 拆分、迁移或灰度切换仍会传导到上游业务逻辑。因此，动态路由机制首先要解决“业务标识”和“Kafka 写入目标”之间的解耦问题。本项目把业务侧输入抽象为 DynamicSinkEvent(routeId, message)，把物理写入目标抽象为 KafkaRouteDestination(kafkaClusterId, bootstrapServers, topicPattern)。因此，逻辑路由模型可以表述为：业务事件先给出 route id，再由 route id 查得物理目标描述，最后由物理目标描述解析出实际的 Topic 集合。这一模型避免了业务逻辑直接感知 Kafka Topic 细节，有利于在 route 配置调整时保持上游代码稳定。

在当前 YAML 配置中，route-a 与 route-b 就是两个典型逻辑 route，它们分

别映射到 `^dynamic-kafka-sink-a$` 和 `^dynamic-kafka-sink-b$`。业务事件只需选择 `route-a` 或 `route-b`，而不必直接写死 Topic 名称。

4.6.2 Map 结构设计

运行时主要维护三张表：`clusterWritersByRouteId` 保存 `route id` 到 `ClusterWriter` 的映射，是最核心的物理写入通道表；`explicitRoutesById` 保存逻辑 `route id` 到 `KafkaRouteDestination` 的映射，表示最新的逻辑配置；`explicitResolvedRouteIdsByLogicalId` 保存逻辑 `route id` 到已解析物理 `route id` 集合的映射，作为 `route` 解析缓存。

这种三表结构的好处是职责清晰：逻辑配置、物理解析结果和底层 `writer` 生命周期彼此解耦。当 `route` 更新到来时，仅需更新前两层结构并在必要时懒加载 `writer`；当快照发生时，则只需遍历实际存在的 `ClusterWriter` 并提取其状态。相比把所有信息塞进一张大表，当前结构更利于维护与调试。

从设计决策看，三表结构也是为了满足第三章的小规模吞吐约束。若每条记录都从逻辑 `route` 重新扫描所有 Topic，动态路由开销会直接进入主写入路径；若把逻辑配置、解析结果和 `writer` 混在一张表中，`route` 更新又容易误关闭仍被其它逻辑 `route` 引用的物理 `writer`。当前做法用 `explicitResolvedRouteIdsByLogicalId` 缓存解析结果，并用 `getReferencedRouteIds()` 判断 `writer` 是否仍被引用，在写入效率和资源清理之间取得折中。

4.6.3 数据分发策略

当前实现包含两种数据分发策略。自动发现路径下，普通元素会被广播写入 `activeRouteIds` 中的所有 `route`，适合“当前所有匹配 Topic 都应接收相同数据”的场景；显式逻辑路由路径下，元素仅写入由 `route id` 解析得到的物理 `route` 集合，适合“按业务类别分流”的场景。

这一设计既能覆盖本文所讨论的“基于模式自动发现”需求，也能支撑“逻辑路由动态更新”的实现目标。但也应看到，自动发现路径的“写向全部活动 `route`”语义比较强，在某些业务场景中可能导致重复分发；后续可进一步引入更细粒度的路由函数来选择单个目标 `route`。

4.7 系统实现细节

4.7.1 核心类设计

核心类设计遵循一个基本原则：动态逻辑尽量集中在外层连接器，Kafka 发送、事务和提交能力尽量复用已有实现。DynamicKafkaSink 是总入口，负责保存构建期配置并在运行时创建 Writer 与 Committer；DynamicKafkaSinkBuilder 负责参数合法性检查，例如必须同时提供 subscriber、metadata service 和 serializer，在 Delivery Guarantee.EXACTLY_ONCE 下还必须提供 transactionalIdPrefix。若用户未显式提供前缀，builder 会自动生成 DynamicKafkaSink-xxxxxxx 形式的默认前缀。

DynamicKafkaSinkWriter 是运行时核心。它内部维护 route 刷新时间、活动 route 集、显式路由表、底层 writer 集合等关键状态，并实现 write(...)、flush(...)、prepareCommit()、snapshotState(...) 和 close() 等完整生命周期方法。底层每个物理 route 对应一个 ClusterWriter，而每个 ClusterWriter 内部实际上又是一个普通 KafkaSink 的 writer 实例。这种“动态外壳 + 静态内核”的设计，是当前原型最重要的工程手法之一。

代码清单 4.3 进一步展示了运行时“刷新 route 并按需创建 writer”的关键算法。可以看到，当前原型并不试图直接改造 Kafka producer 内核，而是对每个解析出的物理 route 单独构造一个普通 KafkaSink writer，再由外层 DynamicKafkaSinkWriter 负责统一调度。这也是本文原型能够在较少改动现有 Connector 基础上快速演化出的重要原因。

Listing 4.3 运行时 route 刷新与底层 writer 惰性创建逻辑

```
private void refreshRouteIfNeeded(boolean force) throws IOException {
    long now = System.currentTimeMillis();
    if (!force) {
        if (metadataRefreshIntervalMs <= 0 && !activeRouteIds.isEmpty()) {
            return;
        }
        if (metadataRefreshIntervalMs > 0 && now <
            nextMetadataRefreshTimestamp) {
            return;
        }
    }

    List<ResolvedRoute> routes = resolveRoutes();
    activeRouteIds = routes.stream()
        .map(route -> route.routeId)
        .collect(Collectors.toSet());
}
```

```
nextMetadataRefreshTimestamp =
    metadataRefreshIntervalMs > 0 ? now + metadataRefreshIntervalMs
    : Long.MAX_VALUE;

for (ResolvedRoute route : routes) {
    clusterWritersByRouteId.computeIfAbsent(
        route.routeId,
        ignored -> createClusterWriter(
            route.routeId,
            route.kafkaClusterId,
            route.topic,
            route.transactionalIdPrefix,
            route.kafkaProducerConfig,
            Collections.emptyList()));
}
cleanupRetiredWriters();
}
```

4.7.2 数据结构设计

数据结构设计同样围绕“动态 route 如何恢复和提交”展开。DynamicKafkaSink WriterState 是 route 级恢复状态对象，包含 route 标识、目标 cluster 与 topic、事务前缀、Producer 配置以及底层 KafkaWriterState。其设计目标不是保存所有业务信息，而是保存“恢复这个 route 的 writer 所必需的最小集合”。

DynamicKafkaCommittable 则是提交阶段的 route 级包装对象。它把底层 KafkaCommittable 与 route 元数据绑定起来，使 DynamicKafkaCommitter 能够先按 route 分组，再为每个 route 懒创建对应的 KafkaCommitter。这样做的好处是不同 route 的提交上下文可以彼此隔离，符合 route 级 writer 的结构。

4.7.3 与 Flink Checkpoint 机制的集成

第三章要求打开 checkpoint-interval-ms=5000 后，route 级状态快照不应明显阻塞本地小规模作业。为满足这一要求，当前 connector 并不是把动态写入逻辑放在 Flink 容错体系之外单独实现，而是直接接入了统一 Sink API 提供的状态快照与恢复链路。最直接的证据是 DynamicKafkaSink 实现了 TwoPhaseCommittingStatefulSink<IN, DynamicKafkaSinkWriterState, DynamicKafkaCommittable>，并同时覆写了 createWriter(...)、restoreWriter(...)、getWriterStateSerializer() 与 createCommitter(...) 等方法。这意味着动态 Sink 在 Flink 看来并不是一个“无状态附加功能”，而是一个能够参与 Checkpoint、恢复和两阶段提交的正式 Sink 组件。

从运行路径看, **Checkpoint** 到来时, `DynamicKafkaSinkWriter.snapshotState(checkpointId)` 会遍历当前所有物理 **route** 对应的底层 **writer**, 对每个 **writer** 调用底层 `KafkaWriterState` 的快照方法, 并把得到的状态列表连同 `routeId`、`kafkaClusterId`、`topic`、`transactionalIdPrefix` 和 **Producer** 配置一起封装为 `DynamicKafkaSinkWriterState`。也就是说, 当前实现不是只保存一个“全局 **route** 集合”, 而是把状态组织成一组 **route** 级恢复单元。

Listing 4.4 动态 Sink 通过 route 级状态接入 Flink Checkpoint

```
@Override
public List<DynamicKafkaSinkWriterState> snapshotState(long checkpointId)
    throws IOException {
    refreshRouteIfNeeded(false);
    List<DynamicKafkaSinkWriterState> states = new ArrayList<>();
    for (ClusterWriter<IN> clusterWriter : clusterWritersByRouteId.values())
    {
        List<KafkaWriterState> kafkaStates = clusterWriter.writer.
            snapshotState(checkpointId);
        if (!kafkaStates.isEmpty()) {
            states.add(
                new DynamicKafkaSinkWriterState(
                    clusterWriter.routeId,
                    clusterWriter.kafkaClusterId,
                    clusterWriter.topic,
                    clusterWriter.transactionalIdPrefix,
                    clusterWriter.kafkaProducerConfig,
                    kafkaStates));
        }
    }
    cleanupRetiredWriters();
    return states;
}
```

当作业故障恢复时, `DynamicKafkaSink.restoreWriter(...)` 会把上一次成功快照中保存的 `DynamicKafkaSinkWriterState` 集合重新传入 `DynamicKafkaSinkWriter` 构造函数。**Writer** 启动后, 先依据恢复态为每个 **route** 重建底层 **KafkaSink writer**, 随后再执行一次 `refreshRouteIfNeeded(true)` 与当前元数据重新对齐。这样一来, 恢复逻辑既保留了 **Checkpoint** 时刻的 **route** 级写入状态, 也允许系统在恢复后继续跟随最新的 **Topic** 拓扑变化。从设计上看, 这正是动态 **connector** 与 **Flink Checkpoint** 机制真正结合的关键点。

4.7.4 事务路径可运行性与 Exactly-once 语义设计

Exactly-once 的设计难点不在于单个 Topic 的事务写入，而在于动态 route 会让事务上下文数量随写入目标变化。如果所有 route 共用同一事务前缀，不同目标之间的提交边界会混在一起；如果完全重新实现事务 producer，又会破坏复用底层 KafkaSink 的设计目标。因此，当前 connector 沿用了 Flink KafkaSink 原有的事务型两阶段提交路径来支撑 EXACTLY_ONCE。首先，在构建阶段，DynamicKafkaSinkBuilder 会在 DeliveryGuarantee.EXACTLY_ONCE 模式下强制要求用户提供 transactionalIdPrefix；否则 builder 直接拒绝构造。这说明当前实现并没有把 Exactly-once 当成“可选附加优化”，而是把它视作需要显式事务前缀才能成立的独立语义模式。

其次，动态 route 语义下的事务前缀并不是所有 route 共用一份，而是通过 buildTransactionalIdPrefix(routeId) 由全局前缀派生出 route 级事务前缀：transactionalIdPrefix + "-" + Integer.toUnsignedString(routeId.hashCode())。这样做的目的是让不同物理 route 的事务上下文彼此隔离，避免多个 route 共享同一个事务标识而造成提交冲突。在真正创建底层 writer 时，DynamicKafkaSinkWriter.createClusterWriter(...) 会把 deliveryGuarantee、transactionalIdPrefix 和 transactionNamingStrategy 一并传给内部构造的普通 KafkaSink，从而复用官方 KafkaSink 已有的事务生产与提交能力。

Listing 4.5 动态 Sink 中 Exactly-once 的 route 级提交路径

```
@Override
public void commit(Collection<CommitRequest<DynamicKafkaCommittable>>
    requests)
    throws IOException, InterruptedException {
    if (deliveryGuarantee != DeliveryGuarantee.EXACTLY_ONCE) {
        return;
    }

    Map<String, List<CommitRequest<DynamicKafkaCommittable>>>
    requestsByRouteId =
        new LinkedHashMap<>();
    for (CommitRequest<DynamicKafkaCommittable> request : requests) {
        requestsByRouteId
            .computeIfAbsent(request.getCommittable().getRouteId(),
                ignored -> new ArrayList<>())
            .add(request);
    }
}
```

```

for (Map.Entry<String, List<CommitRequest<DynamicKafkaCommittable>>>
entry :
    requestsByRouteId.entrySet()) {
    KafkaCommitter kafkaCommitter =
        committersByRouteId.computeIfAbsent(entry.getKey(), ignored
-> /* create */);
    kafkaCommitter.commit(/* route-local KafkaCommittable list */);
}
}

```

再次，在提交阶段，`DynamicKafkaSinkWriter.prepareCommit()` 会把每个 `route` 下底层 `KafkaWriter` 产生的 `KafkaCommittable` 封装成 `DynamicKafkaCommittable`；`DynamicKafkaCommitter.commit(...)` 再按 `routeId` 分组，把提交请求 `fan-out` 到各自的 `KafkaCommitter`。因此，当前 `connector` 的 `Exactly-once` 语义不是一个“全局统一事务”，而是“`route` 级 `writer` 状态 + `route` 级 `committable` + `route` 级 `committer`”共同组成的两阶段提交结构。

最后，需要指出当前实现对 `Exactly-once` 采取了一个保守但合理的工程策略：`cleanupRetiredWriters()` 在 `DeliveryGuarantee.EXACTLY_ONCE` 下会直接返回，即不自动回收退役 `writer`。这一做法表明当前实现已经意识到“事务上下文可能跨 `Checkpoint` 持续存在”，过早关闭退役 `writer` 会给事务边界和恢复一致性带来额外风险。也正因如此，本文后续实验虽已对动态发现和显式 `route` 功能进行了实测，但对 `Exactly-once` 仍然只停留在设计与代码路径说明，尚未覆盖系统性的故障注入与事务恢复验证。

4.7.5 关键流程说明

系统完整流程可概括为：第一，`DynamicJob` 读取 `config.yaml`，生成 `route map` 与测试数据源；第二，`route map` 作为广播更新事件进入作业拓扑；第三，`DynamicKafkaSinkBuilder` 构造 `DynamicKafkaSink`，并把 `pattern`、`metadata service`、`serializer`、投递语义等配置注入其中；第四，`Writer` 启动后执行首次 `refreshRouteIfNeeded(true)`，完成首次 `route` 解析与 `writer` 初始化；第五，运行过程中普通数据事件与 `route` 更新事件共流进入 `Writer`，分别触发自动发现路径或显式 `route` 路径；第六，`Checkpoint` 触发时，`Writer` 对每个物理 `route` 调用底层 `writer` 的 `snapshotState(checkpointId)` 并封装状态；第七，提交阶段 `prepareCommit()` 生成 `route` 级 `committable`，再由 `DynamicKafkaCommitter` 分发提交；第八，故障恢复时，`restoreWriter(...)` 根据恢复态先重建已知 `route writer`，再执行一次强制

刷新以对齐最新元数据。

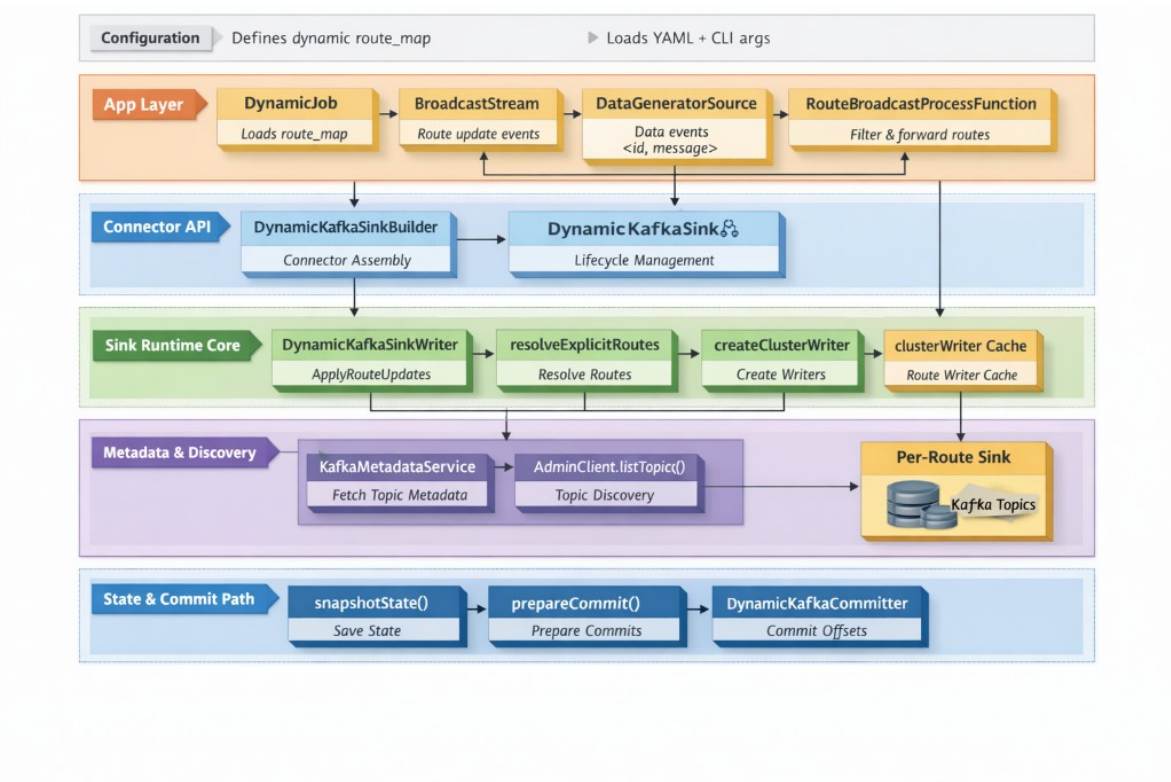


图 4-2 当前原型实现的分层调用关系与状态提交路径（区分运行期流动与组件依赖）

图 4-2 更贴近当前代码，它把应用层、连接器 API 层、Sink 运行时核心、元数据发现层以及状态与提交路径同时展示出来。与上一幅总体结构图相比，该图强调的是 DynamicJob、DynamicKafkaSinkBuilder、DynamicKafkaSinkWriter、AdminClient.listTopics()、snapshotState() 与 prepareCommit() 等真实实现对象之间的依赖与流向，更适合放在实现细节部分作为说明。其中，普通数据事件、route 更新事件、Checkpoint state 和 committable 是运行期产生并流转的对象；builder、metadata service、subscriber、writer 和 committer 之间则是构造、持有或调用依赖。通过这种区分，读者可以分别理解“数据如何写出”和“组件如何协作”两个层次，避免把架构依赖误读为数据传输路径。

4.8 本章小结

本章并非只罗列最终类结构，而是从第三章提出的需求出发，说明了动态 Kafka Sink 的主要设计决策：为了在秒级发现新增 Topic，系统采用 AdminClient 周期轮

询与可配置刷新闻隔；为了让动态路由在小规模场景下接近静态基线吞吐，系统采用 route 解析缓存、writer 懒创建和底层 KafkaSink 复用；为了支持无重启 route 更新，系统利用广播状态传播控制事件，并在 Sink 内部维护逻辑 route 表；为了满足 Checkpoint 协同与事务路径可运行性指标，系统把 writer state、committable 和 committer 都提升为 route 级结构。这些设计共同形成“动态外壳 + 静态内核”的架构，使动态发现、路由更新、writer 管理和状态提交能够在同一套 Flink Sink API 下协同工作。

同时，本章也明确了当前设计中的取舍：全量元数据刷新降低了差分维护复杂度，但在大规模 Topic 场景下可能带来管理面开销；非事务路径已经能够清理退役 writer，而 EXACTLY_ONCE 路径仍采用保守策略以避免破坏事务边界。这些取舍将直接对应第 5 章的发现实时性、吞吐可接受性、小规模扩展性、Checkpoint 协同和事务路径可运行性实验结果。

第五章 实验设计与结果分析

5.1 实验环境与配置

本章实验的被测对象是 connector 模块中的动态 Kafka Sink 实现，尤其是 DynamicKafkaSink、DynamicKafkaSinkWriter、SingleClusterTopicMetadataService 与 DynamicKafkaCommitter 等核心类。为了对这些组件进行可重复验证，本文使用 app 模块提供的若干轻量级作业作为实验载体，但这些作业本身并不是论文的主要交付对象。实验中使用的 Kafka 连接参数、stream_pattern、route_map、并行度与发射速率等配置，已统一整理至附录 1.1；相应运行命令整理于附录 1.2。

数据源方面，原型并未直接接真实业务流，而是使用 DataGeneratorSource 周期性生成测试事件。createRandomCsvSource(...) 会随机选择一个 route id，并生成形如“时间戳, 八位随机数, 随机字符串”的文本消息。这样做的优点是实验可重复、易于控制发送速率，便于观察动态 Sink 在 route 刷新与多目标写入下的行为。

从代码实现角度看，验证载体并不是简单地“持续发送随机字符串”，而是明确把发送速率、route 分布和消息格式都做成了可控变量。代码清单 5.1 展示了数据源的核心实现：当 emit_interval_ms 小于等于 0 时，作业会以尽可能高的速率持续生成数据；否则会通过 RateLimiterStrategy.perSecond(...) 近似控制吞吐。同时，route id 从 YAML 中加载出的 route 列表中均匀随机选择，因此该原型适合用于验证“在多逻辑 route 条件下，动态 Sink 是否能稳定分发与写出”。

Listing 5.1 动态实验作业中的数据生成逻辑

```
private static DataGeneratorSource<DynamicSinkEvent> createRandomCsvSource(  
    long intervalMs, List<String> routeIds) {  
    GeneratorFunction<Long, DynamicSinkEvent> generator =  
        ignored -> {  
            ThreadLocalRandom random = ThreadLocalRandom.current();  
            String routeId = routeIds.get(random.nextInt(routeIds.size()));  
            return DynamicSinkEvent.data(routeId, generateRecord(random));  
        };  
    long recordsPerSecond = intervalMs <= 0 ? Long.MAX_VALUE : Math.max(1L,  
        1000L / intervalMs);  
    return new DataGeneratorSource<>(  
        generator,
```



```
Long.MAX_VALUE,
RateLimiterStrategy.perSecond(recordsPerSecond),
TypeInformation.of(DynamicSinkEvent.class));
}
```

需要说明的是，当前验证载体中的 KafkaSinkBuilder 为了优先完成原型验证，使用的是 DeliveryGuarantee.AT_LEAST_ONCE；而 DynamicKafka Sink 的 builder 设计本身支持 NONE、AT_LEAST_ONCE 与 EXACTLY_ONCE，并可在 EXACTLY_ONCE 模式下依赖底层事务提交路径。因此，本章实验结果主要反映三类内容：动态路由与动态发现机制的功能有效性、connector 与 Flink Checkpoint 的集成表现，以及 route 级 EXACTLY_ONCE 在本地小规模条件下的可运行性。关于 connector 的 Checkpoint 协同机制与事务路径可运行性设计，本文已在第 4 章补充专门说明；本章则进一步给出对应的运行验证结果。

为了使实验与前文需求形成严格对应，本文将第 3 章提出的功能性需求和非功能性指标统一映射到本章实验中。其中，功能性需求关注“机制是否成立”，例如能否发现 Topic、能否建立可写集合、能否按 route 写入；非功能性需求关注“是否达到原型阶段的量化标准”，例如新增 Topic 首次写入延迟是否不超过 3 s、动态方案相对静态基线的吞吐下降是否不超过 5%。表 5-1 给出了需求、实验内容、观测指标和判定标准之间的对应关系。

表 5-1 需求与实验验证对应关系

前文需求	本章实验	观测指标	判定标准
动态 Topic 发现需求	发现实时性实验	新增 Topic 首次写入延迟	最大值不超过 3 s
自动订阅需求	自动订阅验证	新 route 是否创建 writer 并接收数据	新 Topic offset 增长
动态路由需求	显式 route 验证	route-a/route-b 输出分布	目标 Topic 正确且分布均衡
数据处理与状态恢复需求	Checkpoint 验证	成功 Checkpoint 次数与完成时延	连续完成且稳态低于 100 ms
吞吐可接受性	静态基线对照	动态 2-route 相对静态基线吞吐下降	不超过 5%
小规模扩展性	2-route 与 4-route 对照	route 数增加后的吞吐变化	下降不超过 10%
事务路径可运行性	事务路径可运行性实验	Checkpoint 次数与 committed 输出	至少 5 次且双 Topic 有输出

5.2 功能验证实验

5.2.1 动态 Topic 发现验证

动态 Topic 发现验证对应第 3 章的“动态 Topic 发现需求”和“发现实时性”指标。该实验的目标，是确认 **Writer** 能否在作业不重启的前提下识别并使用新出现的 **Topic**，并进一步验证新增匹配 **Topic** 的首次写入延迟是否满足 3 s 的阈值。实验步骤可设计为：首先创建若干满足 `stream_pattern` 的 **Topic**，并启动 `DynamicJob`；然后观察 `DynamicKafkaSinkWriter.refreshRouteIfNeeded(...)` 调用 `resolveRoutes()` 后，`activeRouteIds` 是否被正确填充，`clusterWritersByRouteId` 中是否创建了相应 `route` 的 `ClusterWriter`。在此基础上，再在 **Kafka** 中手动新增符合模式的 **Topic**，等待一个或多个 `discovery interval` 周期，验证新 **Topic** 是否被纳入 `activeRouteIds` 并开始接收消息。

当前原型代码采用周期性全量刷新，因此这一实验关注的不仅是“能否发现”，还包括“多久发现”。只要 `stream-metadata-discovery-interval-ms` 为正值，**Writer** 就会在达到 `nextMetadataRefreshTimestamp` 后再次执行刷新；若该参数为非正值，则首次启动后的 `route` 集合将不再自动变化。因而，实验中需要记录 `discovery interval` 对最终可见时延的影响。

为了对“新增 **Topic** 被发现并开始接收数据”的时间开销进行实测，本文进一步补充了一个专门面向自动发现路径的驱动程序 `AutoDiscoveryJob`。与前文 `DynamicJob` 不同，该作业生成的是 `DynamicSinkEvent.data(null, message)`，即不携带逻辑 `routeId` 的数据记录，从而强制 **Writer** 走 `refreshRouteIfNeeded(...)` 和 `activeRouteIds` 驱动的自动发现路径。实验过程中先创建一个与正则匹配的 `seed Topic` 作为初始可写目标，再在作业运行中连续创建新的匹配 **Topic**，并通过 `kafka-get-offsets` 轮询新 **Topic** 的 `offset`，记录“**Topic** 创建成功时刻”到“该 **Topic** 首次 `offset` 大于 0”之间的时间差。

本次实验共进行了两轮重复测量，每轮连续创建 3 个新 **Topic**，共得到 6 个样本；其余运行参数与第 5.1 节所述环境保持一致。测量结果如表 5-2 所示。

从表 5-2 可以看到，在 `discovery_interval_ms=2000` 的配置下，新增 **Topic** 的首次写入延迟稳定落在约 1.1 到 2.4 秒之间，平均值为 1.53 秒。其中第 2 个样本在两轮实验中都接近 2.4 秒，说明当 **Topic** 创建时刻恰好落在某次刷新之后时，需要等待下一轮元数据刷新才能被纳入 `activeRouteIds`；而其余样本约为 1.1 秒，则说明 **Topic** 创建时刻更接近下一次刷新触发点。该结果与当前实现的时间驱动刷新机

表 5-2 发现实时性验证结果：新增 Topic 首次写入延迟

轮次	Topic 编号	首次写入延迟 / s	首次观测 offset
第 1 轮	1	1.077	14
第 1 轮	2	2.412	23
第 1 轮	3	1.095	8
第 2 轮	1	1.103	14
第 2 轮	2	2.418	24
第 2 轮	3	1.074	10
平均值		1.530	—
中位数		1.099	—
最小值		1.074	—
最大值		2.418	—

制是吻合的，也表明在本地单机环境下，发现实时性指标的主导因素确实是配置的 `discovery interval`，而非 `Kafka` 写入本身。由于最大观测值为 2.418 s，低于第 3 章定义的 3 s 阈值，因此该实验满足发现实时性要求。

5.2.2 自动订阅验证

自动订阅验证对应第 3 章的“自动订阅需求”，关注的是“发现之后，是否能真正写出”。当前实现中，`route` 发现本身并不直接发送消息，真正的写出动作要等到 `clusterWritersByRouteId.computeIfAbsent(...)` 创建出新的 `Cluster Writer` 之后，再由底层 `KafkaSink writer` 负责写出。实验应重点验证两点：一是新 `route` 被发现后，是否确实懒创建了底层 `writer`；二是普通数据事件进入 `write(...)` 后，是否会写入当前所有活动 `route`。

需要如实指出，当前实现已经在非事务路径上补充了退役 `writer` 的清理逻辑：当某个 `route` 不再属于当前活动集合、且也不被显式逻辑 `route` 引用时，`Writer` 会在后续 `flush`、`prepareCommit` 或 `snapshotState` 之后调用清理逻辑将其关闭并移除。因此，自动订阅实验除了验证“新增 Topic 后能否自动扩写”，还可以进一步验证“删除逻辑 `route` 或修改规则后，旧 `route writer` 是否不再长期残留”。不过，对于 `EXACTLY_ONCE` 事务路径，当前实现仍未启用自动回收，因此这一部分仍属于后续工作。结合前述自动发现实验中新 Topic 的 `offset` 由 0 增长到正数这一现象，可以说明新增 `route` 不仅被元数据层发现，也已经经过 `Writer` 解析并建立了实际写入通道，满足自动订阅需求中的“发现后可写”要求。

5.2.3 动态路由验证

动态路由验证对应第 3 章的“动态路由需求”和“数据处理需求”。实验中可先通过 `config.yaml` 配置 `route-a` 与 `route-b`，再由 `DynamicJob` 在作业启动时广播一次 `route map` 更新事件。`BroadcastProcessFunction` 会先把 `route` 写入广播状态，然后把同一条更新事件继续输出给 `Sink`；`Sink` 侧收到更新事件后，`DynamicKafkaSinkWriter.applyRouteUpdates(...)` 把这些逻辑 `route` 写入 `explicitRoutesById`。

在此之后，普通数据事件携带 `route id` 进入 `Sink`，`writeToExplicitRoute(...)` 会先解析逻辑 `route id` 对应的物理 `route`，再把数据写入解析得到的 `Topic` 集合。实验验证可从三个角度展开：第一，发送 `route-a` 的数据是否只会进入 `route-a` 对应的 `Topic`；第二，修改 `route-a` 的 `topic_regex` 后，后续数据是否能在不重启作业的情况下切换到新匹配目标；第三，当某 `route id` 尚未被广播配置时，前置 `BroadcastProcessFunction` 是否会将其过滤掉，避免无效数据进入 `Sink`；第四，当广播事件显式删除某个 `route` 时，广播状态与 `Sink` 内部逻辑 `route` 表是否同步移除该 `route`。

5.2.4 显式路由与静态基线对照验证

本实验同时承担两个作用：一是验证显式逻辑 `route` 能否正确分发到物理 `Topic`，二是为后续“吞吐可接受性”非功能指标提供静态基线对照。对比对象为同一仓库中的 `RegularJob`，它使用传统静态 `KafkaSink` 固定写入单个 `Topic`，不进行元数据刷新、`route` 解析或多 `writer` 管理。因此，该基线适合用于衡量动态机制在小规模显式 `route` 场景下带来的额外吞吐开销。

考虑到短时、低速率实验得到的消息量偏小，不足以作为正文中的主要对照结果，本文重新补充了一组更大样本量的本地实验。为避免历史数据干扰，本次实验使用独立 `Topic` 集合；同时保留一个与 `stream_pattern` 匹配的 `seed Topic`，以满足 `DynamicKafkaSinkWriter` 初始化阶段的 `route` 解析要求。除将数据源发射间隔收紧至 5 ms、单次运行时长提升到约 30 s 外，其余环境仍与第 5.1 节一致。`DynamicJob` 使用附录中的大样本实验配置，加载两条逻辑 `route`：`route-a` 映射到 `^exp-dynamic-c$`，`route-b` 映射到 `^exp-dynamic-d$`；`RegularJob` 则固定写入 `exp-regular-2`。实验完成后，使用 `kafka-get-offsets` 读取各 `Topic` 偏移量作为最终写入条数的近似统计结果，结果如表 5-3 所示。

表 5-3 吞吐可接受性验证结果：动态 2-route 与静态基线对照

方案	目标 Topic 数	运行时间 / s	输出总条数	近似吞吐 / 条 · s ⁻¹
DynamicJob	2	30	5742	191.4
RegularJob	1	30	5776	192.5

进一步查看 DynamicJob 的两个目标 Topic，可以得到 exp-dynamic-c=2812、exp-dynamic-d=2930，分别占动态总写入量的 49.0% 与 51.0%，说明随机生成的数据在两条逻辑 route 之间分布基本均衡，且没有出现某一路由长期偏斜或全部写向同一 Topic 的异常。

从表 5-3 可以得到三点更直接的结论。第一，显式路由路径在当前原型中已经能够稳定工作：在 30 秒、emit_interval_ms=5 的条件下，DynamicJob 能持续向两个物理 Topic 输出总计 5742 条记录。第二，同条件下静态基线写入 5776 条，约为 192.5 条/秒；动态方案写入 5742 条，约为 191.4 条/秒，两者吞吐差距约为 0.59%，说明在本地单机、小规模显式 route 场景下，动态路由引入的额外运行时开销仍然较小。该下降幅度低于第 3 章定义的 5% 阈值，因此满足“吞吐可接受性”要求。第三，两组实验的目标 Topic 集合彼此隔离，说明当前验证载体中的 route 过滤与 connector 内部的 route 解析逻辑在功能上是自洽的。

需要如实说明的是，这一组大样本实测结果主要覆盖了“显式逻辑 route → 物理 Topic”路径，以及与传统静态 KafkaSink 的对照结果；而“自动发现路径在新增 Topic 后的发现时延”则由前文 AutoDiscoveryJob 的单独实验进行量化。

5.2.5 Checkpoint 协同验证

Checkpoint 验证对应第 3 章的“数据处理与状态恢复需求”和“Checkpoint 协同”非功能指标。为了验证当前 connector 是否真正接入了 Flink 的 Checkpoint 链路，本文为 DynamicJob 显式打开了 checkpoint-interval-ms=5000，其余实验环境与第 5.1 节保持一致。该实验的关注点不是吞吐量本身，而是以下三个现象是否同时出现：其一，作业运行期间能够周期性触发 Checkpoint；其二，DynamicKafkaSink Writer.snapshotState(...) 对应的 route 级状态能够被框架正常接纳；其三，Checkpoint 完成后 source 端和 sink 端都能看到“checkpoint completed”的协同日志。

本次验证运行约 22 秒，在日志中观察到连续 5 次成功 Checkpoint，其完成时延分别为 476 ms、11 ms、22 ms、19 ms 和 21 ms；若去除首次冷启动影响，稳态 Checkpoint 完成时延平均约为 18.2 ms。与此同时，日志中可见 Source: id-message-source

与 Source: Collection Source 都收到了“Marking checkpoint as completed”的通知,说明当前验证载体中的 source、广播控制流和动态 Sink 确实共同参与了 Flink 的一致性快照过程。由于稳态 Checkpoint 完成时延约为 18.2 ms,低于第 3 章定义的 100 ms 阈值,该实验满足 Checkpoint 协同要求。

这一结果与第 4 章的设计分析是吻合的:当前 connector 并不是在 Checkpoint 之外单独维护一套外部状态,而是通过 DynamicKafkaSinkWriterState 把 route 级 writer 状态纳入 Flink 原生的快照与恢复机制之中。因此,从正常运行路径的角度看,当前 connector 已经具备与 Flink Checkpoint 机制协同工作的基本能力。

5.2.6 事务路径可运行性验证

事务路径可运行性验证对应第 3 章的同名指标。在上述 Checkpoint 验证基础上,本文进一步对 EXACTLY_ONCE 语义进行了本地小规模验证。实验使用与前文类似的显式 route 驱动方式,但在运行参数中额外指定 delivery-guarantee=EXACTLY_ONCE、checkpoint-interval-ms=5000 和唯一的 transactional-id-prefix。考虑到本地 Kafka broker 对事务超时的限制,验证载体在 EXACTLY_ONCE 模式下使用了更保守的 transaction.timeout.ms=60000,以保证事务生产者能够正常初始化。这一调整不改变 connector 的语义设计,仅用于适配本地实验环境。

表 5-4 事务路径可运行性验证结果

运行模式	Checkpoint 间隔 / s	成功 Checkpoint 次数	输出总条数	近似吞吐 / 条·s ⁻¹
EXACTLY_ONCE	5	5	2092	95.1

进一步查看两个目标 Topic 的 committed offset,可以得到 exp-eo-a=1084、exp-eo-b=1008,分别占总输出的 51.8% 与 48.2%,说明在事务型提交路径下,route 级分发仍保持了与前文显式 route 实验相近的均衡性。更重要的是,本次运行日志中未出现事务前缀冲突、提交失败或事务初始化异常;5 次 Checkpoint 均成功完成,且两个目标 Topic 最终都出现了 committed 输出。这表明当前 connector 的 route 级 DynamicKafkaCommittable 与 DynamicKafkaCommitter 路径在本地单机条件下能够正常工作,其 EXACTLY_ONCE 设计并不只是停留在接口层面,而是已经能够通过小规模实验得到验证。该结果满足第 3 章提出的“至少 5 次 Checkpoint 且两个目标 Topic 均产生 committed 输出”的判定标准。

5.3 非功能需求验证

5.3.1 发现实时性与 Checkpoint 协同验证

本节汇总第 3 章中两个与延迟相关的非功能指标：发现实时性和 Checkpoint 协同。发现实时性关注“新增匹配 Topic 从创建成功到首次写入”的时间，判定标准是不超过 3 s；Checkpoint 协同关注 route 级状态快照是否明显阻塞本地小规模作业，判定标准是稳态完成时延低于 100 ms。

对发现延迟的测量可定义为：从某个新 Topic 在 Kafka 集群中创建成功开始计时，到第一条写入该 Topic 的消息被消费到为止。这一指标主要受 discovery interval、AdminClient.listTopics() 的执行时间和 Flink 任务线程调度影响。例如，在示例配置 discovery interval 为 2000 ms 时，理论上新 Topic 的最早可见时间不会早于下一次刷新触发点。若后续引入缓存或增量元数据策略，可将优化前后的发现时延进行对比。

为了避免本节继续停留在概念层面，本文将前文已经得到的自动发现与 Checkpoint 时延结果汇总为表 5-5。其中，发现实时性数据来自 AutoDiscoveryJob 的两轮共 6 个样本；Checkpoint 协同数据来自 EXACTLY_ONCE 本地验证中的 5 次成功快照。

表 5-5 发现实时性与 Checkpoint 协同验证结果

需求指标	样本数	平均值	中位数/稳态均值	判定结果
发现实时性 / s	6	1.530	1.099	最大 2.418，满足 ≤ 3 s
Checkpoint 协同 / ms	5	109.8	18.2	稳态 18.2，满足 ≤ 100 ms

从表 5-5 可以看到，发现实时性指标稳定落在约 1.1–2.4 秒之间，与 discovery_interval_ms=2000 的配置相吻合，说明当前实现中的时间驱动刷新机制确实主导了新 Topic 的可见时延。而 Checkpoint 完成时延的平均值之所以达到 109.8 ms，主要是首次冷启动快照耗时较高（476 ms）所致；若只观察后续 4 次稳态 Checkpoint，则平均完成时延约为 18.2 ms，说明在本地单机、小规模 route 数量条件下，route 级状态快照并未引入显著的额外阻塞。

5.3.2 吞吐可接受性与小规模扩展性验证

本节对应第 3 章的“吞吐可接受性”和“小规模扩展性”指标。吞吐可接受性采用静态 RegularJob 作为对比对象，判定动态 2-route 方案相对静态基线的吞吐下降是否

不超过 5%；小规模扩展性则比较动态 2-route 与动态 4-route 方案，判定 route 数量增加后的吞吐下降是否不超过 10%。由于当前实现对每个物理 route 都维护一个底层 KafkaSink writer，并且自动发现路径会把普通消息广播写入全部 activeRoute Ids，因此 route 数越多，单条消息触发的实际发送次数越多，CPU、网络与内存开销也会同步增加。实验可通过逐步增加 route 数量、并在固定并行度下提升数据源发射速率，观察系统的稳定吞吐上限。

结合当前代码结构，吞吐实验至少应同步记录三类运行时指标。第一类是输入侧速率，可由 emit_interval_ms 或 recordsPerSecond 直接换算；第二类是动态机制带来的管理成本，例如 clusterWritersByRouteId.size()、activeRouteIds.size() 与 route 解析耗时；第三类是提交与快照成本，可围绕 prepareCommit() 生成的 DynamicKafkaCommittable 数量以及 snapshot State(...) 输出的 route 状态条数进行统计。只有把这几类指标同时观察，才能区分“Kafka 写入瓶颈”和“动态管理逻辑带来的额外成本”。

除了总吞吐量外，还应关注 clusterWritersByRouteId 规模增长对资源占用的影响。当前版本已经在非事务路径上实现了退役 writer 清理，因此历史 route 的累积问题已有所缓解；但在 EXACTLY_ONCE 路径下，由于尚未启用相同的自动回收策略，writer 生命周期管理仍是后续性能优化与一致性验证的重要内容。

表 5-6 汇总了本章中与吞吐相关的几组实测结果。为保证可比性，除 EXACTLY_ONCE 验证组外，其余实验均在本地单机、parallelism=1、emit_interval_ms=5 的条件下运行约 30 秒。4-route 动态组的输出总量按 Topic offset 增量统计，因为其中两个 Topic 复用了前一组 2-route 实验所创建的主题。

表 5-6 吞吐可接受性与小规模扩展性验证结果

方案	运行模式	route 数	输出总条数	近似吞吐	判定用途
静态基线	AT_LEAST_ONCE	1	5776	192.5	基线
动态写入 (2-route)	AT_LEAST_ONCE	2	5742	191.4	吞吐可接受性
动态写入 (4-route)	AT_LEAST_ONCE	4	5763	192.1	小规模扩展性
动态写入 (2-route)	EXACTLY_ONCE	2	2092	95.1	事务路径可运行性

从表 5-6 可以得到三点观察。第一，在 AT_LEAST_ONCE 条件下，静态基线、2-route 动态和 4-route 动态三组实验的吞吐都稳定在约 191–193 条/秒范围内，其中动态 2-route 相比静态基线下降约 0.59%，低于第 3 章定义的 5% 阈值，说明在本地单机、小规模 route 数量场景下，动态 route 管理尚未成为主导瓶颈。第二，4-route 动

态组并未明显低于 2-route 动态组，这表明在当前实验规模下，route 数量从 2 增长到 4 还没有把写放大成本放大到足以压低整体吞吐的程度；按近似吞吐计算，4-route 结果甚至略高于 2-route 结果，因而满足“下降不超过 10%”的小规模扩展性指标。第三，当切换到 EXACTLY_ONCE 后，吞吐下降到约 95.1 条/秒，约为 AT_LEAST_ONCE 动态 2-route 结果的一半，这说明事务初始化、Checkpoint 协同和 route 级提交确实带来了更高的运行成本，也与 EXACTLY_ONCE 语义更强调一致性而非极致吞吐的设计目标相符。

5.3.3 非功能验证结果分析

综合本节结果可以看到，在本地单机和小规模 route 数量条件下，当前 connector 的性能瓶颈仍主要体现在事务提交路径，而不是 2 到 4 个 route 之间的 writer 管理差异。这也说明，如果后续要评估更大规模动态目标集合下的扩展边界，仍需要在分布式环境中进一步补充更高 route 数量、更高并行度和更长运行时间的系统性测试。

5.4 对比实验与结果分析

5.4.1 静态 Kafka Sink 基线对比

静态基线采用本仓库中的 RegularJob，其核心特征是由应用层在构建作业时直接指定固定 Topic，运行期间 Topic 不会自动更新。与其相比，动态 Sink 的主要优势不在于单条消息的绝对发送速度，而在于 Topic 变化时无需停机重启。因此，本文将静态方案作为“吞吐可接受性”的对比对象：在相同本地环境、相同发送速率和相近运行时间下，比较 RegularJob 固定单 Topic 写入与 DynamicJob 2-route 动态写入的近似吞吐差异。

该静态基线在本仓库中已有对应实现，即 RegularJob。从代码清单 5.2 可以看到，基线方案只需要把固定 Topic 直接写入 KafkaRecordSerializationSchema，不会涉及元数据轮询、逻辑 route 更新或多 writer 管理。也正因为如此，静态方案的实现复杂度和运行时开销都更低，但代价是每当目标 Topic 集合发生变化时，必须通过重新构建或重启作业来完成切换。

Listing 5.2 静态 KafkaSink 基线方案的核心构造逻辑

```
KafkaSink<String> sink =  
    KafkaSink.<String>builder()  
        .setBootstrapServers(bootstrapServers)  
        .setRecordSerializer()
```

```
KafkaRecordSerializationSchema.<String>builder()  
    .setTopic(topic)  
    .setValueSerializationSchema(new  
SimpleStringSchema())  
    .build())  
.setDeliveryGuarantee(DeliveryGuarantee.AT_LEAST_ONCE)  
.build();
```

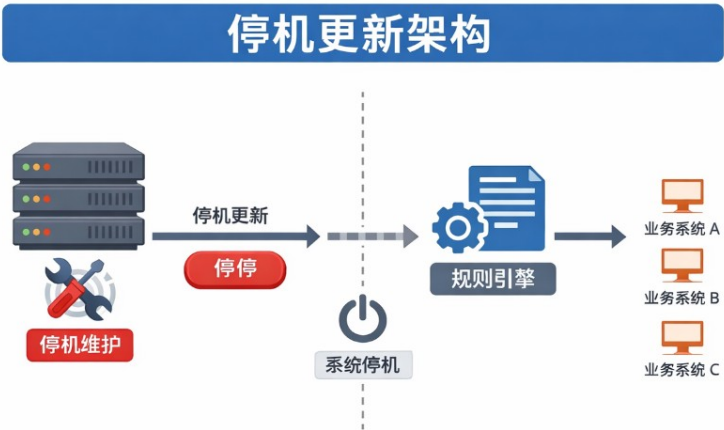


图 5-1 传统停机更新架构示意

图 5-1 给出了传统停机更新架构的典型流程：当规则或目标系统发生变化时，通常需要先停机维护、停止写入，再更新规则引擎或输出配置，最后恢复业务系统的下游接入。这一模式虽然容易理解，但会把“配置变化”直接转化为“业务链路停顿”，因此非常适合作为本文动态方案的静态对照基线。

结合表 5-6 可知，静态基线的吞吐为 192.5 条/秒，动态 2-route 方案的吞吐为 191.4 条/秒，下降约 0.59%。这一结果说明：在拓扑稳定、只写单 Topic 的场景下，静态方案仍具有更低实现复杂度；但在本文关心的动态 Topic 场景下，动态方案以很小的本地吞吐代价换取了无重启发现和 route 更新能力。

5.4.2 Source 侧正则订阅与 Sink 侧动态写入对比

若仅与 Source 侧正则订阅能力相比，动态 Sink 的创新点不在“正则本身”，而在“把正则匹配结果落到写入路径上”。Source 侧 FLIP-246 关注的是消费订阅集合与分区变化，Sink 侧则需要处理 Producer 配置、Topic 绑定、writer 状态快照与 route 级提

交等问题，二者虽然都依赖元数据发现，但内部代价结构并不相同。因此，本课题的对比分析应强调：动态 Sink 不是简单把 Source 的正则订阅搬到写端，而是在写入端增加了一层 route 级 writer 管理与提交逻辑。

5.4.3 本文方案优势分析

结合当前原型，可以把本文方案的优势概括为三点。第一，面向 Topic 频繁增删的场景，系统能够在 discovery interval 驱动下自动扩写新的目标，不需要人工修改作业配置并重新发布。第二，通过 DynamicSinkEvent 与 KafkaRouteDestination 的组合，系统把“业务路由”与“Kafka 物理目标”解耦，使 route 调整可以通过广播更新在运行期完成。第三，动态 Sink 并未重写整套 Kafka 提交逻辑，而是把每个 route 下沉为一个普通 KafkaSink writer，再通过 route 级状态与 committable 包装复用现有实现，降低了原型开发复杂度。

与此同时，当前实现的成本也十分明确：需要周期性访问 AdminClient；route 越多，writer 管理越复杂；自动发现路径默认对所有活动 route 广播写入，可能放大写放大开销；route 回收与 Exactly-once 的系统化验证仍待完成。因此，本文方案更适合作为“动态写入机制原型与可行性验证”，后续还需要在工程化层面继续打磨。

5.5 本章小结

本章围绕第 3 章提出的功能性与非功能性需求逐项进行了验证。功能方面，自动发现实验验证了新增 Topic 可以被识别并开始接收数据，显式 route 实验验证了逻辑 route 到物理 Topic 的正确分发，Checkpoint 协同与事务路径可运行性实验验证了 route 级状态和提交路径能够接入 Flink 运行时。非功能方面，发现实时性最大观测值为 2.418 秒，满足 3 s 阈值；动态 2-route 相对静态基线吞吐下降约 0.59%，满足 5% 阈值；2-route 到 4-route 未出现吞吐下降，满足小规模扩展性指标；稳态 Checkpoint 完成时延约 18.2 ms，满足 100 ms 阈值；事务路径可运行性实验完成 5 次 Checkpoint 且两个目标 Topic 均有 committed 输出。

整体来看，当前实验结果与前文需求和设计形成了闭环，说明该动态 Kafka Sink 原型在本地单机、小规模 route 数量条件下具备可行性。与此同时，本文实验仍属于原型验证，未覆盖大规模 Topic 集合、分布式高并发、复杂故障注入和长期资源占用等场景，这些内容仍需作为后续工程化评估方向。

第六章 总结与展望

6.1 研究工作总结

本文围绕“基于 Flink 的动态 Kafka Topic 发现与订阅机制”这一主题，完成了从问题提出、相关工作分析，到系统设计、实现与验证的完整过程。与仅停留在概念层面的方案不同，本文工作已经落到可运行的连接器代码：在 connector 模块中实现了 `DynamicKafkaSink`、`DynamicKafkaSinkBuilder`、`DynamicKafkaSinkWriter`、`SingleClusterTopicMetadataService`、`DynamicKafkaSinkWriterState`、`DynamicKafkaCommittable` 与 `DynamicKafkaCommitter` 等核心类；app 模块主要作为实验与验证载体，用于驱动 connector 暴露出的动态发现、显式 route 更新与静态基线能力。

从实现路径看，本文并未完全推翻现有 `KafkaSink`，而是采取“动态外壳 + 静态内核”的思路：外层由 `DynamicKafkaSinkWriter` 负责 route 解析、元数据刷新与多 writer 协调，内层每个物理 route 仍复用标准 `KafkaSink` 的 writer、状态与提交逻辑。这种设计既满足了本文关于“逻辑路由与物理路由分离”的设计目标，也与开题阶段对“基于正则表达式实现动态发现、并与 Flink Checkpoint 机制结合”的问题界定基本一致。当前实现已经能够在不重启作业的情况下接收 route 更新事件、自动解析匹配 Topic 并完成多目标写入，形成了一套完整可运行的动态 Sink 方案。

进一步而言，本文的价值并不只在于“实现了一个能运行的 demo”，更在于把问题拆分为可操作的几个关键环节：一是以元数据服务和正则匹配为基础完成动态目标发现；二是以逻辑 route 与物理 Topic 解耦完成运行期路由更新；三是以 route 为单位组织 writer、状态和提交路径，使动态行为尽可能兼容现有 `KafkaSink` 的实现框架。这种拆分方式使课题既具备工程可落地性，也具备较清晰的方法论结构。

6.2 存在的不足

尽管当前 connector 已具备原型可运行性，但距离更成熟的工程化方案仍有若干差距。首先，当前元数据发现依赖 `AdminClient.listTopics()` 的周期性轮询，在 Topic 规模扩大后会带来额外管理面开销，并且 `discovery interval` 决定了发现时延下界。其次，当前实现能够新增 route writer，但尚未在 route 失活时及时回收 cluster

WritersByRouteId 中的旧 **writer**，因而在长时间运行或频繁变更场景下，资源占用可能持续累积。再次，当前验证载体主要使用的是 `AT_LEAST_ONCE`，虽然动态 Sink 设计上支持 `EXACTLY_ONCE`，但尚未结合复杂故障场景完成系统化验证。

此外，当前自动发现路径默认会把普通记录广播写入所有 `activeRouteIds`，这在某些“多目标复制”场景中是合理的，但在更一般的业务分流场景下可能造成不必要的写放大。日志、监控指标、异常治理与配置版本控制等工程特性，也仍需进一步加强。这些不足与中期报告中“元数据访问开销、实时性、一致性、工程化能力仍需完善”的结论相互呼应。

在本文最新实现中，逻辑 **route** 的删除语义已经补充完成，非事务路径下的退役 **writer** 清理也已落地，这使 **route** 生命周期管理相比中期阶段更加完整；但 `EXACTLY_ONCE` 路径上的安全回收、事务边界验证与更精细的资源治理仍未完全解决。需要强调的是，这些不足并不否定原型本身的研究意义，而是说明当前实现更适合作为“动态 Sink 机制的验证性实现”。也就是说，本文已经证明了动态发现、动态路由与 **route** 级状态封装在 Flink Kafka Sink 场景下具有可行性，但距离一个大规模、长期稳定运行的生产级方案，还需要在资源治理、边界条件和异常恢复方面继续完善。

6.3 未来工作展望

未来的改进可从四个方向推进。第一，优化元数据访问机制，在保持动态发现能力的同时降低轮询成本，例如引入缓存、差分刷新或更细粒度的发现策略，以缩短 Topic 变化到可写之间的时延。第二，完善 **writer** 生命周期管理，在 **route** 失活后安全回收对应 **writer**，并研究更合理的 **route** 级资源上限控制策略。第三，围绕 `EXACTLY_ONCE` 路径设计更系统的故障恢复实验，验证 **route** 级状态恢复、事务前缀管理、提交重试与幂等性边界。第四，进一步强化工程化能力，包括指标埋点、日志可观测性、异常分类处理、配置热更新版本管理以及更丰富但不改变 **connector** 核心语义的验证载体。

在论文工作层面，后续还需要结合完整实验数据补充第 5 章中的定量分析，例如 `discovery interval` 对发现延迟的影响、**route** 数量增长对吞吐与资源占用的影响，以及动态方案与静态 `KafkaSink` 的对比结果。通过这些补充，可使本文从“原型验证与可行性说明”进一步提升为“具有更强实证支撑的毕业设计成果”。

总体而言，本文已经完成了毕业设计从“选题论证”到“原型落地”的关键阶段。后续只要继续围绕实验数据补充、实现细节打磨和论文语言精修推进，就可以使整篇论文在工程深度、论证完整性和学术表达上进一步趋于成熟。

参考文献

- [1] KREPS J, NARKHEDE N, RAO J. Kafka: a distributed messaging system for log processing[J]. Proceedings of the NetDB, 2011.
- [2] CARBONE P, KATSIFODIMOS A, EWEN S, et al. Apache Flink: Stream and batch processing in a single engine[J]. IEEE Data Eng. Bull., 2015, 38(4): 28-38.
- [3] Apache Flink. Apache Kafka Connector[EB/OL]. [2026-03-01]. <https://nightlies.apache.org/flink/flink-docs-stable/docs/connectors/datastream/kafka/>.
- [4] Apache Flink. Writing Data Using the Unified Sink API (Kafka 连接器文档) [EB/OL]. [2026-03-01]. <https://nightlies.apache.org/flink/flink-docs-stable/docs/connectors/datastream/kafka/>.
- [5] Apache Flink Community. FLIP-246: Dynamic Kafka Source[EB/OL]. [2026-03-01]. <https://cwiki.apache.org/confluence/pages/viewpage.action?pageId=217389320>.
- [6] ORHIDI M. FLIP-515: Dynamic Kafka Sink[EB/OL]. [2026-03-01]. <https://cwiki.apache.org/confluence/display/FLINK/FLIP-515:+Dynamic+Kafka+Sink>.
- [7] MA H, MA Y, LI J, et al. An automated method for solving Configuration of Distributed Messaging Systems: DMGA-PSO algorithm[J]. Expert Systems with Applications, 2025.
- [8] 王懿宸. Astraea Partitioner: 基于过去全局資訊實現 Kafka 叢集節點動態負載平衡[D]. 成功大學, 2022.
- [9] 吳昱緯. 以 LogP 效能模型特徵化 Apache Kafka[D]. 成功大學, 2021.
- [10] QIU Y, et al. Towards Fine-Grained Scalability for Stateful Stream Processing Systems[J]. arXiv preprint arXiv:2503.11320, 2025.
- [11] LI H, LI J, DUAN X, et al. Energy-aware scheduling and two-tier coordinated load balancing for streaming applications in apache flink[J]. Future Generation Computer Systems, 2025.
- [12] TOSHNIWAL A, et al. Storm@twitter[C]//Proceedings of the 2014 ACM SIGMOD International Conference on Management of Data. 2014: 147-156.
- [13] NOGHABI S A, et al. Stateful Scalable Stream Processing at LinkedIn[J]. Proceedings of the VLDB Endowment, 2017.
- [14] KATSIFODIMOS A, SCHELTER S. Apache Flink: Stream Analytics at Scale[C]//Tutorial at ACM SIGMOD. 2016.
- [15] SAKET S, CHANDELA V, KALIM M D. Real-time Event Joining in Practice With Kafka and Flink[J]. arXiv preprint arXiv:2410.15533, 2024.
- [16] Apache Kafka. Apache Kafka Documentation: Message Delivery Semantics[EB/OL]. [2026-03-01]. <https://kafka.apache.org/documentation/#semantics>.

实验配置与复现命令（附录 1）

1.1 实验配置样例

为了避免在正文中反复插入过多配置细节，本文将实验所用的典型配置整理于本附录。这些配置仅用于驱动 connector 的验证，并不改变 connector 本身的设计与语义。

1.1.1 显式 route 与静态基线实验配置

Listing 1.1 用于显式 route 与静态基线实验的配置样例

```
dynamic:
  cluster_id: default-cluster
  topic: exp-dynamic
  stream_pattern: "^exp-wjh.*$"
  bootstrap_servers: localhost:9092
  emit_interval_ms: 20
  parallelism: 1
  discovery_interval_ms: 2000
  route_map:
    route-a:
      cluster_id: default-cluster
      bootstrap_servers: localhost:9092
      topic_regex: "^exp-dynamic-a$"
    route-b:
      cluster_id: default-cluster
      bootstrap_servers: localhost:9092
      topic_regex: "^exp-dynamic-b$"
regular:
  topic: exp-regular
  bootstrap_servers: localhost:9092
  emit_interval_ms: 20
  parallelism: 1
```

1.1.2 发现实时性实验配置

发现实时性实验没有依赖显式 route map，而是使用 AutoDiscoveryJob 生成不携带 routeId 的数据记录，并通过 stream_pattern 驱动 Dynamic KafkaSinkWriter.refreshRouteIfNeeded(...) 进行 route 刷新。对应命令中使用的关键参数为：bootstrap-servers=localhost:9092、

```
cluster-id=default-cluster、stream-pattern=^exp-auto-.*$、  
emit-interval-ms=50、parallelism=1、discovery-interval-ms=2000。
```

1.2 复现实验命令

本文实验主要通过以下几类命令复现：

Listing 1.2 论文编译与字数统计命令

```
./run.sh pdf  
./run.sh word
```

Listing 1.3 根目录脚本提供的本地 Kafka/Flink 运行入口

```
./run.sh setup  
./run.sh run dynamic  
./run.sh run regular
```

Listing 1.4 发现实时性实验的核心执行命令示意

```
java -cp app/target/dynamic-kafka-sink-app-1.0-SNAPSHOT.jar \  
org.apache.flink.dynamic.sink.job.AutoDiscoveryJob \  
--bootstrap-servers localhost:9092 \  
--stream-pattern '^exp-auto-.*$' \  
--cluster-id default-cluster \  
--emit-interval-ms 50 \  
--parallelism 1 \  
--discovery-interval-ms 2000
```

Listing 1.5 Checkpoint 协同与事务路径可运行性验证命令示意

```
java -cp app/target/dynamic-kafka-sink-app-1.0-SNAPSHOT.jar \  
org.apache.flink.dynamic.sink.job.DynamicJob \  
--bootstrap-servers localhost:9092 \  
--stream-pattern '^exp-wjh-eo.*$' \  
--cluster-id default-cluster \  
--emit-interval-ms 10 \  
--parallelism 1 \  
--discovery-interval-ms 2000 \  
--checkpoint-interval-ms 5000 \  
--delivery-guarantee EXACTLY_ONCE \  
--transactional-id-prefix thesis-eo-demo \  
--config-file config.experiment.exactly_once.yaml
```

学术论文和科研成果目录

本科毕业设计期间，作者暂无已发表论文及科研成果。

致 谢

感谢那位最先制作出博士学位论文 $\text{L}^{\text{A}}\text{T}_{\text{E}}\text{X}$ 模板的物理系同学!

感谢 William Wang 同学对模板移植做出的贡献!

感谢 @weijianwen 学长开创性的工作!

感谢 @sjtug 对 0.10 及之后版本的开发和维护工作!

感谢所有为模板贡献过代码的同学们, 以及所有测试和使用模板的各位同学!

感谢 $\text{L}^{\text{A}}\text{T}_{\text{E}}\text{X}$ 和 SJTUThesis, 帮我节省了不少时间。



RESEARCH ON DYNAMIC KAFKA TOPIC DISCOVERY AND SUBSCRIPTION MECHANISM BASED ON FLINK

With the continuous development of real-time data infrastructure, Apache Kafka and Apache Flink have become a common technical combination for event-driven systems, log processing, monitoring, and real-time analytics. Kafka provides durable and high-throughput distributed logs, while Flink provides stateful stream processing, checkpoint-based fault tolerance, and flexible external output through the unified Sink API. In production systems, upstream services write events into Kafka, Flink jobs consume and transform these streams, and the results are written back to Kafka topics or other downstream systems. This architecture works well when the topology is stable, but it becomes less flexible when downstream topics change frequently. In multi-tenant or evolving business systems, Kafka topics may be created by tenant, date, region, business domain, or migration stage. A long-running Flink job may therefore face a changing set of write targets.

The conventional Flink Kafka Sink is mainly configured statically. Users usually specify the target topic, bootstrap servers, serialization schema, and delivery guarantee before the job is submitted. After deployment, these write targets normally remain fixed. If the topic set changes, operators often need to stop the job, modify configuration, resubmit the job, and wait for recovery. This increases operational cost and complicates consistency management. The source side has explored dynamic topic subscription through pattern-based discovery, but the sink side has additional challenges. It must discover target topics, bind records to physical topics, create and reuse writers, snapshot route state, and coordinate commit semantics. 

This thesis studies a dynamic Kafka topic discovery and subscription mechanism based on Flink, with a focus on the Sink side. The goal is to allow a Flink job to discover matching Kafka topics and adjust its write paths without being restarted. The work is implemented in a Maven project with a `connector` module and an `app` module. The `connector` module contains the main contribution, including `DynamicKafkaSink`, `DynamicKafkaSinkBuilder`, `DynamicKafkaSinkWriter`, `SingleClusterTopicMetadataService`, `DynamicKafkaSinkWriterState`, `DynamicKafkaCommittable`, and `DynamicKafkaCommitter`. The `app` module provides validation

jobs such as `DynamicJob`, `AutoDiscoveryJob`, and `RegularJob`. These jobs generate test records, distribute route updates, and compare dynamic writing with a static baseline.

The main design idea is the separation between logical routes and physical Kafka topics. Instead of forcing business records to carry a fixed topic name, the system allows records to carry a logical route identifier. This identifier is mapped to a `KafkaRouteDestination`, which contains a Kafka cluster id, bootstrap servers, and a topic pattern. The writer resolves the logical destination into one or more physical routes by querying Kafka metadata and matching topic names with a regular expression. This design allows the physical target of a business route to change while keeping upstream business logic stable. It also supports fan-out semantics when one logical route matches multiple physical topics, although this behavior may introduce write amplification and therefore must be clearly documented.

The overall architecture can be described as a dynamic shell over a static core. The static core is the existing Kafka writing capability: after a physical route has been resolved, serialization, producer sending, flushing, and transaction handling are delegated to an inner `KafkaSink` writer. The dynamic shell is implemented mainly by `DynamicKafkaSinkWriter`. It is responsible for metadata refresh, route resolution, writer creation, route cache management, state snapshot, and commit preparation. This design avoids rewriting a complete Kafka producer layer and reuses the mature fault-tolerance and transaction path of the Kafka sink implementation. As a result, the dynamic behavior is concentrated at the route management layer, while the stable write path remains close to the original connector.

The metadata discovery layer is implemented by `SingleClusterTopicMetadataService`. It uses `Kafka AdminClient.listTopics()` to obtain the currently visible topic set from a Kafka cluster. Each topic is converted into a `Kafka Stream`, and the discovery result is filtered by `StreamPatternSubscriber` according to a configured regular expression. The refresh interval is controlled by `stream-metadata-discovery-interval-ms`. With this mechanism, discovery delay is mainly bounded by the configured interval, the execution time of the metadata request, and Flink task scheduling delay. The writer then maintains two paths. The automatic discovery path writes ordinary records to all currently active routes. The explicit logical route path resolves a record's route id into physical routes and writes only to the

corresponding target set. Route update events can add, overwrite, or remove route mappings, and the resolved cache is invalidated accordingly.

State and commit management are essential for compatibility with Flink. The dynamic writer stores route-level recovery units instead of a single global state. `DynamicKafkaSinkWriterState` wraps route metadata, producer configuration, and the underlying `KafkaWriterState`. During snapshot and recovery, these states are used to rebuild known route writers and then align them with the current Kafka topology. For commit management, `DynamicKafkaCommittable` wraps the underlying `KafkaCommittable`, while `DynamicKafkaCommitter` groups commit requests by route and delegates them to per-route Kafka committers. Thus, the transaction path is organized by route-level states, committables, and committers rather than one global transaction across all routes.

The thesis defines measurable requirements before presenting experiments. Functionally, the system should support dynamic topic discovery, automatic subscription of writable routes, dynamic routing through logical route ids, and stable processing with serialization, Kafka writing, checkpointing, and recovery. Non-functionally, the prototype should satisfy concrete criteria in a local validation environment. Under `discovery_interval_ms=2000`, the first write to a newly created matching topic should occur within three seconds. In the explicit two-route scenario, throughput loss compared with the static `RegularJob` baseline should be less than five percent. When route count grows from two to four, throughput should not drop by more than ten percent. With `checkpoint_interval_ms=5000`, steady checkpoint completion time should stay below one hundred milliseconds. For the transaction path, a small-scale `EXACTLY_ONCE` run should complete at least five checkpoints and produce committed output on both target topics.

The discovery timeliness experiment uses `AutoDiscoveryJob`. It generates records without a logical route id, starts with a seed topic, and creates new matching topics while the job is running. By polling Kafka offsets, the experiment measures the delay between topic creation and the first positive offset. Six samples are collected in two rounds. The observed first-write delays range from about 1.074 seconds to 2.418 seconds, with an average of 1.530 seconds. These results are consistent with the configured two-second discovery interval and satisfy the three-second target.

The explicit route and throughput experiments use `DynamicJob`. Two logical routes are loaded from YAML and mapped to different topic patterns. In a thirty-second local run with `emit_interval_ms=5`, the dynamic two-route job writes 5742 records. The two target topics receive 2812 and 2930 records, corresponding to 49.0 percent and 51.0 percent of the dynamic output. The static baseline implemented by `RegularJob` writes 5776 records, or about 192.5 records per second. The dynamic two-route job reaches about 191.4 records per second, a decrease of about 0.59 percent, which is below the five-percent threshold. A four-route dynamic run achieves about 192.1 records per second, showing no observable throughput degradation in the small-scale setting.

Checkpoint coordination is validated with `checkpoint-interval-ms=5000`. Five successful checkpoints are observed, with completion times of 476 ms, 11 ms, 22 ms, 19 ms, and 21 ms. Excluding the cold-start checkpoint, the steady average is about 18.2 ms, below the one-hundred-millisecond threshold. The transaction path is further validated in `EXACTLY_ONCE` mode. With a unique transactional id prefix and `transaction.timeout.ms=60000`, five checkpoints complete successfully, no commit failure is observed, and two target topics both receive committed output. This shows that the route-level committable and committer path is executable in a small local environment.

The implementation and experiments demonstrate that extending `KafkaSink` with dynamic topic discovery, logical route mapping, route-level writer management, and route-level state and commit wrappers is feasible. Compared with rewriting a full Kafka producer layer, this approach reuses existing `KafkaSink` behavior and keeps dynamic logic in the connector shell. The prototype can discover new topics, route records according to logical ids, write to multiple topics, participate in checkpointing, and execute a small-scale transaction path. It also has limitations. Metadata discovery relies on periodic polling, the automatic discovery path may cause write amplification, retired writers are not automatically reclaimed in `EXACTLY_ONCE` mode, and the transaction prefix strategy still deserves stronger collision-resistant design. Future work should focus on metadata efficiency, transaction-safe cleanup, observability, fault injection, and larger distributed validation.